
Lectures Datascience for Economics

Jan Boone

August 2026

Preamble

There are two versions of this document:

- the interactive html version which contains the interactive marimo apps
- and the static pdf version which does not contain interactive apps but shows links to these apps

The html version uses the ReadTheOrg theme and the pdf version the Eisvogel latex template.

Marimo

In the lectures we use marimo notebooks. If you follow these lectures as a Tilburg University student, marimo is available at jupyterlab server.

Information on how to install marimo on your own computer can be found [here](#). You can also use molab.

Introduction

The goal of these lectures is to build intuition for statistical and machine-learning concepts through simulation. Rather than deriving results analytically, we generate data ourselves, repeat an experiment thousands of times, and read the answer directly from the resulting distribution. This approach — sometimes called **statistical hacking** — serves two purposes: you always know exactly where your data comes from, and you sharpen your Python skills along the way.

Each lecture starts from a concrete economic or statistical question, introduces the Python tools needed to answer it through simulation, and then explains the code in detail. Interactive marimo apps let you experiment with the key parameters and see the effects immediately.

The lectures build on each other. We begin with foundational ideas in statistics (distributions of estimators, bootstrapping, OLS) and move toward machine-learning techniques such as regularisation, neural networks, and treatment-effect estimation.

Contents

Preamble	1
Marimo	1
Introduction	2
Lecture 1: Statistical Hacking — Distributions and Bootstrapping	6
Learning Objectives	6
Introduction	6
Apps	7
Distribution of an estimator	7
Bootstrap hypothesis test	8
Python Code Explanation	8
Simulating sampling distributions	9
Distribution of the OLS slope	10
Bootstrapping	12
OLS from scratch: the optimization approach	13
Summary	15
Review Questions	15
Question 1: Distribution of the sample standard deviation	16
Hint	16
Question 2: OLS with correlated predictors	16
Question 3: Ridge regression and variable selection	17
Hint	17
Question 4: One-sided vs two-sided bootstrap test	17
Question 5: Bootstrap confidence interval	17
Hint	18
Lecture 2: Variable selection and causality	19
Learning Objectives	19
Introduction	19
Apps	21
Significance and variable selection	21
Fork, pipe, and collider	22
Python Code Explanation	22
Multiple testing	22
Simulating fork, pipe, and collider	24

Summary	30
Review Questions	31
Question 1: False discovery rate	31
Question 2: Fork with partial effect	31
TODO Question 3: Costly data acquisition	32
TODO Question 4: Treatment effects	32
Hint	35
Lecture 3: Tensors, broadcasting, indexing, matrix operations	37
Learning Objectives	37
Introduction	37
Apps	38
Interactive matrix operations	38
Gradient descent for linear regression	38
Python Code Explanation	38
Tensors and shape	38
Slicing and indexing	39
Broadcasting	40
Matrix operations	41
Gradient descent from scratch	43
Summary	46
Review Questions	47
Question 1: Slicing and fancy indexing on panel data	47
Question 2: Sampling distributions without loops	47
Question 3: Multicollinearity and the matrix inverse	48
Question 4: Batched OLS with <code>np.matmul</code>	48
Lecture 4: Overfitting and underfitting	49
Learning Objectives	49
Introduction	50
Apps	50
Polynomial regression: over-fitting and under-fitting	51
Ridge and lasso regularisation	51
Python Code Explanation	51
Polynomial features and train/test split	51
Ridge regression: shrinking coefficients	53
Lasso regression: coordinate descent and sparsity	54
Tuning λ with cross-validation	56

Bias–variance decomposition	57
Why lasso gives sparsity but ridge does not	59
Regularisation path	61
Information criteria	61
Summary	63
Review Questions	63
Lecture 5: Neural network	63
why relu?	64
Perceptron	66
Multi-layer perceptron	72
neural network graphical illustration	79
Neural network with tensorflow	79
using history to check for over/under fitting	79
example neural network: confusion matrix	80
Economic Neural Networks	82
Nonlinear Phillips curve	82
Solving for equilibrium	85
Model	85
Closed-form solution (for verification)	86
Goal	86
Course takeaway: simulation as a bridge to machine learning	89
Bibliography	90

Lecture 1: Statistical Hacking — Distributions and Bootstrapping

Skills you learn in this lecture: simulating sampling distributions, visualising histograms, implementing bootstrapping and OLS from scratch using Python and numpy.

Learning Objectives

After completing this lecture you should be able to:

- explain why simulating a distribution is a useful alternative to analytical statistical theory
- reproduce the Central Limit Theorem in Python by simulating sample means
- describe what the sampling distribution of an OLS slope looks like and what determines its spread
- implement a bootstrap permutation test to compare two group means without distributional assumptions
- define the loss function for OLS in Python and minimise it numerically
- explain the difference between ridge and lasso penalisation and implement both as modifications to the OLS loss

Introduction

In your statistics and econometrics courses you have seen analytical results: the sample mean follows an approximately normal distribution by the Central Limit Theorem; the OLS slope has a t-distribution under certain assumptions. These results are elegant but limited. Modern machine-learning models are so complex that no such closed-form distributions exist for their estimated parameters. So how do you assess uncertainty?

The answer is **simulation**. You write the statistical problem in Python, run it thousands of times, and read the distribution from the resulting histogram. This lecture introduces this way of thinking through three closely linked ideas:

1. **Distributions of estimators** — how much does the sample mean or OLS slope vary from sample to sample?
2. **Bootstrapping** — how can you test a hypothesis when you have only one sample and no knowledge of the underlying model?
3. **OLS from scratch** — how does the optimisation problem behind OLS work, and how do ridge and lasso extend it?

Because you generate all the data yourself, you always know the true answer and can check whether the simulation recovers it. That is what makes this approach so useful for building intuition.

The statistical results in this lecture are probably (hopefully) not new to you. What we want to show to you is the new way (simulation or “hacking”) in which to understand these results. Later on in the lectures there are topics where we do not have analytical results; then simulation is the only option to gain intuition.

If you like this approach to statistics, you can read a free book (Downey 2014) or watch YouTube lectures from Allen Downey and Jake VanderPlas.

Apps

This lecture has two interactive apps. The first shows how the sampling distribution of the mean and the OLS slope change as you adjust the sample size and noise level. The second demonstrates bootstrap hypothesis testing: you control the effect size and watch whether the test detects it.

Distribution of an estimator

In your statistics classes you have probably done hypothesis testing on a sample mean. Then there was “something” with a normal distribution and “another thing” with $\sqrt{n}$ where n was the sample size. Depending on the math content of your course, you have seen these results proved (or not). Here we will explain what these results are and what the intuition is by programming/simulating them. To illustrate, we calculate the mean of n draws out of a uniform distribution; then we do this, say, 10,000 times. Thus we have 10,000 means and we can show the histogram of these means. What does this histogram look like; put differently, what is the distribution of these means? What is the standard deviation of these 10,000 means?

The clear advantage of an analytic proof is that it shows that the result is (always) true. Simulations cannot prove anything. But simulations can help your intuition and sometimes more than a formal proof. Because we actually simulate the concepts, you can literally look at them. For many students the distribution of a mean is a rather abstract concept. We plot this distribution with simulations making it more concrete. The standard error is then the standard deviation of this distribution. Similarly, the distribution of a slope in an OLS regression may sound vague, but when we simulated the different samples and estimate the slope 10,000 times we can plot these 10,000 slopes.

The app below has two parts. In the first part you choose a sample size n and a number of simulations N . Each simulation draws a sample of size n from the uniform distribution on $[0, 1]$ and records the sample mean. The histogram of the N means shows the sampling distribution of the estimator (the mean); the red curve is the theoretical normal distribution from the Central Limit Theorem. When are these two distributions close and when do they seem different?

In the second part you choose a true slope β_1 , a noise level σ , and sample size dimensions; i.e. the sample size n of the data and the number of repetitions N of our simulation. The app then runs N OLS regressions and plots both the histogram of N estimated slopes and a fan of 50 fitted regression lines. This illustrates that uncertainty about the slope translates directly into uncertainty about where the regression line lies — and that the uncertainty is largest far from the mean of x .

https://janboone.github.io/datascience-marimo/lecture1_distributions.html

Bootstrap hypothesis test

Above we assume that we know the relevant theoretical (underlying) distributions and draw samples from it. But what if you have a real world dataset and no information on the distribution from which this data is drawn? How can the n sample with N repetitions work then? We illustrate this with data from two groups A and B and we want to test whether the population means of the two groups are the same. That is, the null hypothesis is: $\mu_A = \mu_B$.

Suppose we observe data from two groups A and B and want to know whether their (population) means are the same. In the app, group B is generated as $B = A \times \delta$ where $\delta \leq 1$, so a smaller δ means a larger difference between the means of the A and B groups. When $\delta = 1$ there is no difference; the null hypothesis is true.

The left panel shows the raw samples and their means. The right panel shows the bootstrap null distribution: the distribution of mean differences you would see if both groups were actually drawn from the same population. The observed difference is marked in red; the shaded tails are the fraction that exceed it in magnitude — that fraction is the p-value.

Drag δ from 0.80 toward 1.00 to watch the p-value rise above 0.05 as the effect becomes too small to detect with the given sample size.

https://janboone.github.io/datascience-marimo/lecture1_bootstrap.html

Python Code Explanation

In this section we explain the Python code for the apps above. Open a marimo notebook alongside this document and type the code yourself; do not just copy and paste. Getting errors and fixing them is the fastest way to learn.

We focus here on modelling the statistical intuitions not on the interactive elements in the marimo notebooks.

Simulating sampling distributions

Import libraries and set parameters

We use `numpy` for numerical computation and `matplotlib` for plotting.

```
import numpy as np
import matplotlib.pyplot as plt

sample_size = 10          # n observations per sample
N_simulations = 10_000    # N repetitions
```

Generate the data

The key idea: instead of drawing one sample, we draw `N_simulations` samples at once by creating a 2D array (matrix). Each row is one sample (of n draws); each column is one observation within that sample.

```
samples = np.random.uniform(0, 1, (N_simulations, sample_size))
means = samples.mean(axis=1)
```

`np.random.uniform(low, high, size)` draws uniformly distributed random numbers. The argument `(N_simulations, sample_size)` gives a matrix with `N_simulations` rows and `sample_size` columns.

`.mean(axis=1)` takes the mean *across columns* (axis 1), so we get one mean per row; one per simulation. Compare this to `.mean(axis=0)` which would average across rows.

Plot the histogram

Setting `density=True` normalizes the histogram so that the total area equals 1, which makes it comparable to a probability density function. The `alpha` parameter controls transparency (0 = invisible, 1 = opaque).

For a sample drawn from `Uniform[0,1]`, the population mean is $\mu = 0.5$ and the population variance is $\sigma^2 = 1/12$. The standard error of the sample mean is therefore $SE = 1/\sqrt{12n}$. By the Central Limit Theorem, the sampling distribution of the mean is approximately $\mathcal{N}(0.5, SE^2)$ for large n .

```

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(means, bins=60, density=True, alpha=0.6, color="steelblue",
      ↪ label="Simulated means")
ax.set_xlabel("Sample mean")
ax.set_ylabel("Density")
ax.set_title(f"Sampling distribution of the mean (n={sample_size})")
se = 1.0 / np.sqrt(12 * sample_size)
x = np.linspace(0.2, 0.8, 300)
pdf = (1.0 / (se * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x - 0.5) / se) ** 2)
ax.plot(x, pdf, "r-", lw=2, label=f"Theoretical N(0.5, {se:.4f}^2)")
ax.legend()

```

We compute the normal PDF manually using the formula $f(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$. Overlaying it on the histogram checks whether the Normal approximation is good for the chosen `sample_size`.

Distribution of the OLS slope

To simulate the distribution of an OLS slope, we need to generate n draws of (x, y) pairs, fit a regression to this sample, record the slope, and repeat this N times.

Generate data with broadcasting

```

N_simulations = 1000
n_sample_size = 20
true_slope = 0.5
true_intercept = 1.0
noise = 0.5

x = np.random.normal(0, 1, (N_simulations, n_sample_size))
eps = np.random.normal(0, noise, (N_simulations, n_sample_size))
y = true_intercept + true_slope * x + eps

```

Here `true_slope * x` multiplies every element of the matrix `x` by the scalar `true_slope`. This is called **broadcasting**: numpy automatically extends scalar operations to match the shape of the array. In a later lecture we come back to broadcasting in numpy because it is very powerful feature.

Compute OLS slopes

The OLS slope formula for a simple regression is $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$. We can compute this for all simulations at once:

```
x_c = x - x.mean(axis=1, keepdims=True)
y_c = y - y.mean(axis=1, keepdims=True)

slopes_hat = (x_c * y_c).sum(axis=1) / (x_c ** 2).sum(axis=1)
```

The argument `keepdims=True` ensures the mean has shape `(N_simulations, 1)` instead of `(N_simulations,)`, so that subtracting it from the `(N_simulations, sample_size)` matrix `x` broadcasts correctly.

We plot the distribution of estimated slopes (histogram) and the theoretical distribution. If you do not know the theoretical distribution of OLS slopes; do not worry: we use simulations in these lectures, not theoretical results. If you do know the distribution of OLS slopes: why do we plot a normal distribution and not a Student t distribution?

```
# Histogram of estimated slopes
plt.hist(slopes_hat, bins=50, density=True, alpha=0.65, color="coral",
         label="Estimated slopes  $\hat{\beta}_1$ ")
plt.axvline(true_slope, color="k", lw=2, linestyle="--",
            label=f"True slope = {true_slope:.1f}")
x_hist_range = np.linspace(slopes_hat.min() - 0.2, slopes_hat.max() + 0.2,
                           300)
slope_se = noise / np.sqrt(n_sample_size)
normal_slope_pdf = (
    (1.0 / (slope_se * np.sqrt(2 * np.pi)))
    * np.exp(-0.5 * ((x_hist_range - true_slope) / slope_se) ** 2)
)

plt.plot(x_hist_range, normal_slope_pdf, "b-", lw=2,
         label=f"Theoretical N({true_slope:.1f}, {slope_se:.3f}^2)")

plt.xlabel("Estimated slope  $\hat{\beta}_1$ ")
plt.ylabel("Density")
plt.title("Distribution of OLS slope estimator")
plt.legend(fontsize=8)
plt.tight_layout()
```

Bootstrapping

Generate two groups

We generate the A sample from a normal distribution. The B sample is $\delta \in \langle 0, 1 \rangle$ from the A sample. Hence we know that $\mu_B \leq \mu_A$ (with probability 1.0). Note that we generate these A and B samples once. Above we generated these samples N times; but here we generate the repetitions in a different way.

```
sample_size = 50
delta = 0.95      # B = A * delta; delta < 1 means B is lower than A
A = np.random.normal(10, 1, sample_size)
B = A * delta
observed_difference = A.mean() - B.mean()
print(f"The observed difference equals {observed_difference:.2f}")
```

Combine the samples

Under the null hypothesis (H_0 : both groups come from the same distribution); in other words, the group labels are arbitrary. We combine the two samples:

```
AB = np.concatenate([A, B])
```

Run the permutation test

We randomly relabel the combined sample N_{boot} times and record the difference in means each time. Using `np.random.choice` we generate a matrix with labels for the permutation. To see what this function does, check out `np.random.choice(5, size=(15, 10), replace=True)`. Also try `replace=False` and see what happens.

For each of the N repetitions (which are in rows, `axis = 0`), we calculate the difference in means between the labelled A and labelled B samples.

```
N_boot = 10000
rng = np.random.default_rng()

perm_indices =
    ↪ np.random.choice(sample_size, size=(N_boot, 2*sample_size), replace=True)
```

```
boot_diff = (AB[perm_indices[:, :sample_size]].mean(axis=1)
             - AB[perm_indices[:, sample_size:]].mean(axis=1))
```

AB[perm_indices[:, :sample_size]] uses **fancy indexing**: for each of the N_boot rows we select sample_size elements of AB according to the permuted indices. In a later lecture we come back to fancy indexing.

Compute the p-value

```
p_value = np.mean((boot_diff) >= (observed_difference))
print(f"p-value: {p_value:.4f}")
```

(boot_diff) >= (observed_difference) produces a boolean array; taking its mean gives the fraction of bootstrap differences that are at least as extreme as the observed one. That fraction is the (one-sided) p-value.

OLS from scratch: the optimization approach

The final topic of this lecture (with statistical concepts you have probably seen before) is OLS. Above we use the analytical expression for the slope of a simple regression (i.e. only one independent variable): $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$. The same result can be obtained by minimizing the mean squared error loss function directly. This is useful because the loss function can later be extended with penalty terms to give ridge or lasso.

Generate data

```
from scipy import optimize

sample_size = 500
slope1 = -3.0
slope2 = 2.0
constant = 1.0
noise = 1.5
```

```
x1 = np.random.normal(0, 1, sample_size)
x2 = np.random.normal(0, 1, sample_size)
y = constant + slope1 * x1 + slope2 * x2 + noise * np.random.normal(0, 1,
    ↪ sample_size)
```

Define the OLS loss

The OLS estimator minimizes the mean squared error:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (w_0 + w_1 x_{1i} + w_2 x_{2i} - y_i)^2$$

```
def loss_ols(w):
    predictions = w[0] + w[1] * x1 + w[2] * x2
    return np.mean((predictions - y) ** 2)
```

Minimize the loss

```
result = optimize.minimize(loss_ols, x0=[0.0, 0.0, 0.0])
print("Estimated coefficients:", result.x)
```

`optimize.minimize` starts from the initial guess `x0` and finds the values of `w` that minimize `loss_ols`. You should recover values close to `constant=1`, `slope1=-3`, and `slope2=2`.

Ridge and lasso penalties

Ridge and lasso add a penalty on the coefficients to the loss function. This discourages the model from assigning large coefficients to variables that have little predictive power; a problem that becomes serious when you have many variables.

Ridge uses a squared penalty:

```
lam = 1.0

def loss_ridge(w):
    predictions = w[0] + w[1] * x1 + w[2] * x2
```

```
mse = np.mean((predictions - y) ** 2)
penalty = lam * np.sum(w[1:] ** 2)  # do not penalise the intercept
return mse + penalty
```

Lasso uses an absolute-value penalty:

```
def loss_lasso(w):
    predictions = w[0] + w[1] * x1 + w[2] * x2
    mse = np.mean((predictions - y) ** 2)
    penalty = lam * np.sum(np.abs(w[1:]))
    return mse + penalty
```

The key difference: lasso can shrink coefficients exactly to zero (performing variable selection), while ridge only shrinks them toward zero. Experiment with different values of `lam` to see the effect. We will come back to ridge and lasso regression in the next lecture.

Summary

In this lecture we introduced statistical simulation as a practical complement to analytical theory. We illustrated the Central Limit Theorem by plotting sampling distributions of the mean, and showed that OLS slope estimates also have a distribution that widens with noise and narrows with sample size.

We then implemented bootstrap hypothesis testing: by repeatedly reshuffling a combined sample under the null hypothesis, we obtained a p-value without any distributional assumption. Finally, we implemented OLS as a numerical optimization problem and extended it to ridge and lasso by adding penalty terms to the loss function.

Looking ahead: In the next lecture we consider the issue of variable selection and causal interpretation of coefficients.

Review Questions

Test your understanding. For each question, write the code in your own `marimo` notebook.

TODO review questions:

- plot std of mean(x) against $\sigma/\sqrt{n}$
- test signif. of slope with bootstrapping; how can we bootstrap under null hypoth. that slope equals 0
- plot the separate ols lines from the first app

Question 1: Distribution of the sample standard deviation

Above we looked at the distribution of the sample *mean*. The sample *standard deviation* s also has a distribution.

a. Using the same setup as in section Simulating sampling distributions, simulate the distribution of s across `N_simulations` samples. Plot its histogram. b. What is the probability that $s < 0.28$? Compute this from the simulated distribution. c. How does the shape of the distribution of s change as you increase `sample_size`? Is it symmetric?

Hint

- Replace `means = samples.mean(axis=1)` with `stds = samples.std(axis=1, ddof=1)`. The argument `ddof=1` gives the unbiased estimator.
- `np.mean(stds < 0.28)` gives the fraction of simulated standard deviations below 0.28.

Question 2: OLS with correlated predictors

In the data-generating process above, x_1 and x_2 are independent. Now generate correlated predictors:

```
x1 = np.random.normal(0, 1, sample_size)
x2 = 0.8 * x1 + np.sqrt(1 - 0.8**2) * np.random.normal(0, 1, sample_size)
y = constant + slope1 * x1 + 3 * np.random.normal(0, 1, sample_size)
```

a. Minimise the OLS loss with this correlated x_2 . How do the estimated coefficients change? b. Simulate the distribution of the estimated slope on x_1 for 1000 repetitions. How does correlation between x_1 and x_2 affect the spread of this distribution?

Question 3: Ridge regression and variable selection

Use the setup from but now generate five predictors, of which only two have a non-zero true coefficient:

```
n = 200
X = np.random.normal(0, 1, (n, 5)) # five predictors
true_w = np.array([1.0, -3.0, 0.0, 0.0, 0.0])
y = X @ true_w + np.random.normal(0, 1, n)
```

a. Minimise the OLS loss. Do the estimated coefficients for the irrelevant predictors come out exactly zero? b. Minimise the ridge loss with $\lambda = 1$. How does the result differ? c. Minimise the lasso loss with $\lambda = 0.1$. Are any coefficients exactly zero?

Hint

- The loss function now takes a vector w of length 5 and computes $X @ w$ for the predictions.
- For lasso the gradient of $|w|$ is undefined at zero, so `scipy.optimize.minimize` may not converge cleanly. Try the method 'L-BFGS-B' which handles bounds and subgradients better.

Question 4: One-sided vs two-sided bootstrap test

The bootstrap test we implemented is **two-sided**: we ask whether $|D_{\text{boot}}| \geq |D_{\text{obs}}|$. In some situations we have a directional hypothesis — for instance, we expect $A > B$.

a. Adapt the p-value computation to compute a **one-sided** p-value that tests whether the mean of A is specifically *larger* than the mean of B . b. For a fixed $\delta = 0.95$ and $n = 50$, run the test 200 times and record the one-sided p-value each time. What fraction of tests reject at the 5% level? Is this what you expect?

Question 5: Bootstrap confidence interval

Instead of a hypothesis test, you can also use the bootstrap to construct a confidence interval for the difference in means.

a. From the 10,000 bootstrap differences you computed, extract the 2.5th and 97.5th percentiles. This gives a 95% bootstrap confidence interval for the difference. b. Does this interval contain zero when $\delta = 1$? What about when $\delta = 0.90$?

Hint

- `np.percentile(boot_diff, [2.5, 97.5])` gives the two endpoints of the interval.
 - This is the **percentile** bootstrap CI. It is conceptually simple but there are more accurate variants (BCa, studentised) that you can explore if interested.
-

Lecture 2: Variable selection and causality

Skills you learn in this lecture: recognizing that significance is not a variable selection criterion, reading and reasoning with directed acyclic graphs (DAGs), simulating fork, pipe, and collider structures in Python to see how controlling for variables changes OLS estimates.

Learning Objectives

After completing this lecture you should be able to:

- explain why significance is not a valid criterion for selecting variables
- define a directed acyclic graph (DAG) and use it to describe a causal structure
- distinguish fork, pipe, and collider structures and state which variables should (not) be controlled for in each case
- simulate fork, pipe, and collider data in Python using numpy
- compute OLS coefficients for simple and multiple regressions from scratch and interpret how they change when a control variable is added or removed

Introduction

In your statistics and econometrics courses you have learned that OLS measures correlations. A regression coefficient tells you how Y changes with changing values for X , but it does not say whether there is a causal relationship. In economics, causal questions matter enormously: if you advise the government to adopt policy X because it will raise Y , you need to be sure that X actually *causes* Y , not just that the two are correlated.

Two common but mistaken shortcuts that researchers use when they move from correlations to causal claims are the following.

First, some people select control variables based on statistical significance: add a variable, check whether it is significant, keep it if yes. The xkcd cartoon in Figure [ref:fig:significance_xkcd](#) illustrates the absurdity of this approach. If you test 20 unrelated variables at the 5% level, you expect one to appear significant purely by chance. The first app in this lecture lets you explore this multiple testing problem directly.

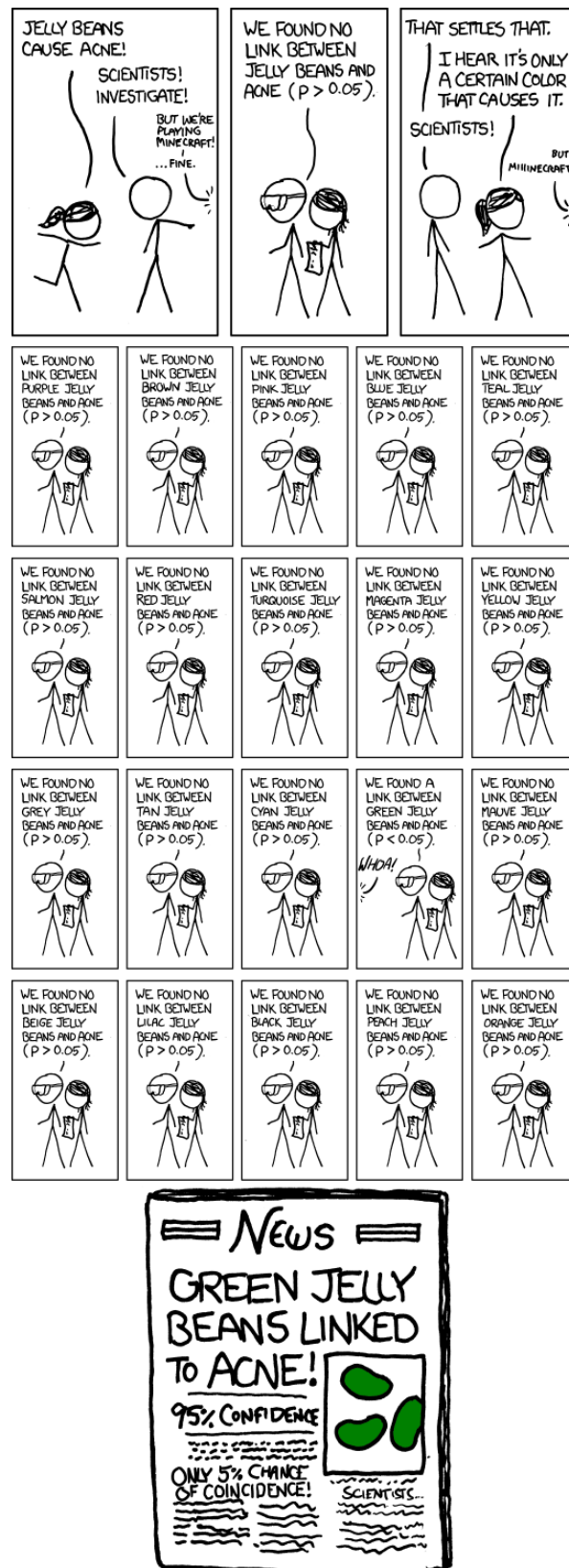


Figure 1: Significance cannot be used for variable selection (in case you need an explanation).

Second, and more subtly, even if you have the right variable in mind as a control, adding it to a regression can either fix a problem or make it dramatically worse; depending on the causal structure. A third variable Z can play three fundamentally different roles:

- the **fork** — Z causes both X and Y ; failing to control for Z creates a spurious correlation between X and Y
 - a simple example is the correlation between the consumption of ice cream and the number of people drowned on a given day; can you guess what Z is here?
- the **pipe** — Z mediates the causal effect of X on Y ; controlling for Z blocks the causal path you are trying to measure
 - an economic example: you want to know the effect of education on income; should you control for (un)employment?
- the **collider** — both X and Y cause Z ; controlling for Z induces a spurious correlation between X and Y where none existed
 - suppose running X and cycling Y skills are uncorrelated in the student population; let Z denote membership of the university's elite duathlon team. What is the correlation between X and Y conditional on Z ? Hint: consider a team member with low X .
 - as the collider is the least intuitive of the three, below we analyze two more examples; one with simulated and one with real data.

Each of these three structures is represented by a directed acyclic graph (DAG): a diagram with variable names as nodes and arrows showing the direction of causation. The second app lets you switch between the three DAG types and see how the OLS coefficient of X changes when you add Z to the regression.

Apps

This lecture has two interactive apps. The first demonstrates the multiple testing problem. The second shows the fork, pipe, and collider structures and lets you observe how adding a control variable Z to the regression changes the estimated effect of X on Y .

Significance and variable selection

All k variables in this app are generated independently of Y — none has a true effect. Nevertheless, some will appear significant at the chosen α level just by chance. The histogram shows the distribution of t-statistics across all k regressions; the red lines mark the critical values. The bar chart on the

right compares the expected number of false discoveries ($\alpha \times k$) with the number actually observed. Increase k to 100 or 200 to see how many spurious findings accumulate.

https://janboone.github.io/datascience-marimo/lecture2_significance.html

Fork, pipe, and collider

Select a DAG type from the dropdown and watch the bar chart on the right. The two bars show the OLS coefficient of X from a bivariate regression ($Y \sim X$) and from a multiple regression that also includes Z ($Y \sim X + Z$). The t-statistics are shown in the chart title. The callout below the chart gives the interpretation: for the fork, controlling for Z is the right thing to do; for the pipe and the collider, it is not.

Increase the effect strength γ to make the bias more visible, or reduce the sample size to see how estimation uncertainty interacts with the bias.

https://janboone.github.io/datascience-marimo/lecture2_causality.html

Python Code Explanation

In this section we explain the key ideas behind the two apps. Open a marimo notebook alongside this document and type the code yourself.

We focus on the statistical logic — how to simulate each causal structure and how to compute OLS coefficients from scratch. The interactive elements (sliders etc.) in the apps are added on top of this code.

Multiple testing

Simulate many “null variables”

We generate k variables that have no effect on Y and run a multiple regression of Y on all k variables. The matrix $\mathbf{X}$ has shape $(n, k + 1)$: n observations in our (simulated) sample, a column of ones for the intercept and one column per variable.

```
import numpy as np

n, k = 100, 50
Y = np.random.normal(0, 1, n)
X = np.random.normal(0, 1, (n, k))
```

Compute t-statistics for the multiple regression

OLS gives $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$ and the covariance matrix of $\hat{\beta}$ is $\hat{\sigma}^2 (\mathbf{X}^\top \mathbf{X})^{-1}$ where $\hat{\sigma}^2 = \text{RSS} / (n - k - 1)$ and RSS denotes Residual Sum of Squares.

For ease of reference we provide the shape of the vector/matrix in a comment on the same line. In a later lecture we come back to matrix operations like multiplication @, transform T, solve and inv.

```
Xmat = np.column_stack([np.ones(n), X])           # (n, k+1)
beta = np.linalg.solve(Xmat.T @ Xmat, Xmat.T @ Y) # (k+1,)
resid = Y - Xmat @ beta                          # (n,)
sigma2 = np.sum(resid ** 2) / (n - k - 1)         # df = n - k - 1
cov_beta = sigma2 * np.linalg.inv(Xmat.T @ Xmat)  # (k+1, k+1)
se = np.sqrt(np.diag(cov_beta))                  # (k+1,) standard errors
t_stats = beta[1:] / se[1:]                      # (k,); slopes only
```

The vector `t_stats` holds one t-statistic per variable, each testing $H_0 : \beta_j = 0$. Under the null, each $t_j \sim t_{n-k-1}$; for $n = 100$ and $k = 50$ the critical value at 5% is approximated by 1.96.

```
n_significant = np.sum(np.abs(t_stats) > 1.96)
print(f"{n_significant} out of {k} variables are 'significant' at 5%")
print(f"Expected: {0.05 * k:.1f}")
```

Every significant result here is a false discovery — by construction.

Distribution of false discoveries across repetitions

One run gives one draw from the distribution of false discoveries. To see the full distribution we repeat the simulation $N = 10,000$ times and record the number of significant variables for each draw.

```
N = 10_000
counts = np.empty(N, dtype=int)

for i in range(N):
    Yi = np.random.normal(0, 1, n)
    Xi = np.random.normal(0, 1, (n, k))
    Xmat_i = np.column_stack([np.ones(n), Xi])
    beta_i = np.linalg.solve(Xmat_i.T @ Xmat_i, Xmat_i.T @ Yi)
    resid_i = Yi - Xmat_i @ beta_i
    s2_i = np.sum(resid_i ** 2) / (n - k - 1)
```

```
se_i      = np.sqrt(np.diag(s2_i * np.linalg.inv(Xmat_i.T @ Xmat_i)))[1:]
counts[i] = int(np.sum(np.abs(beta_i[1:] / se_i) > 1.96))
```

Because each t-test is approximately independent and rejects with probability $\alpha = 5\%$ under the null, the count of significant variables follows a Binomial(k, α) distribution with mean $k\alpha$ and variance $k\alpha(1 - \alpha)$.

```
from scipy.stats import binom
import matplotlib.pyplot as plt

alpha = 0.05
xs = np.arange(counts.max() + 2)

sim_probs = np.array([(counts == x).mean() for x in xs])
binom_probs = np.array([binom.pmf(x, k, alpha) for x in xs])

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(xs - 0.2, sim_probs, width=0.35, alpha=0.75,
       color="steelblue", label=f"Simulated ({N:,} runs)")
ax.bar(xs + 0.2, binom_probs, width=0.35, alpha=0.75,
       color="coral", label=f"Binomial({k}, {alpha})")
ax.set_xlabel(f"Number of 'significant' variables out of k = {k}")
ax.set_ylabel("Probability")
ax.set_title("Distribution of false discoveries – simulation vs. Binomial")
ax.legend()
plt.tight_layout()
```

Simulating fork, pipe, and collider

For all three structures it is convenient to define a helper function that computes OLS coefficients for a simple or multiple regression using numpy linear algebra. As above, the matrix $\mathbf{X}$ starts with a column of ones (the intercept) and the predictor columns. The estimated coefficients are given by $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$.

```
def ols(Y, *Xs):
    """Return OLS coefficient vector [intercept, beta_1, ...]."""
    Xmat = np.column_stack([np.ones(len(Y))] + list(Xs))
    return np.linalg.solve(Xmat.T @ Xmat, Xmat.T @ Y)
```

The fork

Z causes both X and Y ; there is no direct effect from X to Y .

As mentioned in the app: if there is an effect between two variables, we assume the effect equals γ .

Here we set $\gamma = 1$.

```
n = 1000
gamma = 1.0
Z = np.random.normal(0, 1, n)
X = gamma * Z + np.random.normal(0, 1, n)    # Z causes X
Y = gamma * Z + np.random.normal(0, 1, n)    # Z causes Y

coef_without_z = ols(Y, X)                    # Y ~ X: biased
coef_with_z     = ols(Y, X, Z)                # Y ~ X + Z: correct

print(f"Y ~ X:      coef of X = {coef_without_z[1]:.3f}")
print(f"Y ~ X + Z: coef of X = {coef_with_z[1]:.3f}")
```

The bivariate regression picks up the spurious correlation created by Z . Adding Z removes it.

The pipe

X causes Z which causes Y ; the causal effect of X on Y is fully mediated by Z .

```
X = np.random.normal(0, 1, n)
Z = gamma * X + np.random.normal(0, 1, n)    # X causes Z
Y = gamma * Z + np.random.normal(0, 1, n)    # Z causes Y

coef_without_z = ols(Y, X)                    # correct
coef_with_z     = ols(Y, X, Z)                # incorrect

print(f"Y ~ X:      coef of X = {coef_without_z[1]:.3f} (true total effect ~
  ⇨ {gamma**2:.2f})")
print(f"Y ~ X + Z: coef of X = {coef_with_z[1]:.3f} (mechanism is blocked)")
```

An economic example: the effect of schooling on income is partly mediated through employment: schooling helps you to find employment. Controlling for employment makes schooling look less relevant, but the real effect exists; it works through employment.

The collider

X and Y are independent. Both cause Z . Conditioning on Z induces a spurious negative correlation between X and Y .

```
X = np.random.normal(0, 1, n)
Y = np.random.normal(0, 1, n)
Z = gamma * X + gamma * Y + np.random.normal(0, 1, n)

coef_without_z = ols(Y, X)      # correct
coef_with_z     = ols(Y, X, Z)  # wrong

print(f"Y ~ X:      coef of X = {coef_without_z[1]:.3f} (correct, near
      ↪ zero)")
print(f"Y ~ X + Z: coef of X = {coef_with_z[1]:.3f} (spurious negative!)")
```

The intuition: if we hold $Z = \gamma X + \gamma Y$ fixed, then a higher X must be offset by a lower Y to keep Z constant. So within any slice of Z , X and Y are negatively correlated which is a pure artifact of conditioning.

Finally, we consider the collider in two situations where there is correlation between X and Y : first with simulated and then with real data.

collider bias with simulated data

Many people have the intuition that adding a control variable to a regression (especially when it turns out to be significant) is a good idea. However, this intuition is wrong if it is applied without thinking. The pipe and collider above are examples where this reasoning is incorrect. Here we consider a collider in a more elaborate example.

We use here the education example by McElreath (2020) to illustrate this point (see Richard McElreath's lecture 6). Consider the educational achievements of three generations in one family: the grandparent, the parent and the (grand)child. We are interested in the effect of the grandparent's education on the educational achievement of the grandchild.

Let's denote the child's educational achievement by Y (the variable we are interested in) and the grandparent's education X . We want to understand the effect of X on Y . Clearly there is the following path: more educated grandparent leads to more educated parent (Z) which, in turn, leads to more educated grandchild. The thing we are interested in is whether there is a direct effect from the grandparent on the child, *controlling* for the parent's education.

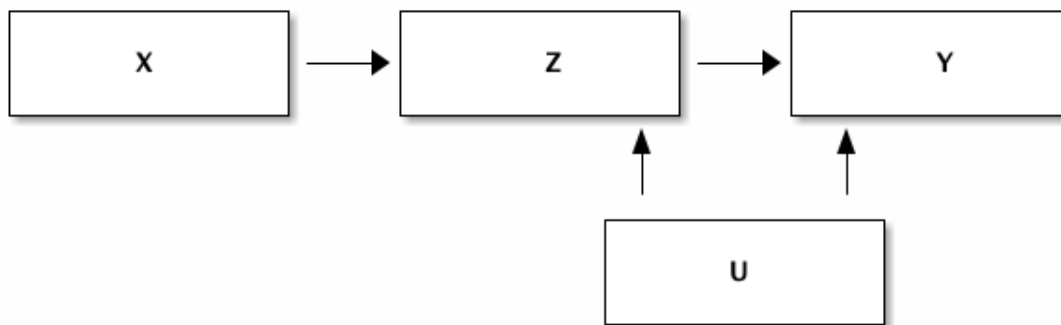


Figure 2: A collider in a more complicated context.

We know that the parent's education has a positive effect on the child's achievement. A more educated parent will tend to stimulate their children more, can help with homework etc. In the data that we generate below, we assume that $Z = X + \dots$ and $Y = Z + \dots$. That is, we assume that the parent's education feeds one-to-one into the child's education.

Is it the case that on top of this effect there is a separate grandparent effect? E.g. because the grandparent is babysitting and a more educated grandparent reads Hamlet with the 5 year old grandchild. That is fun and can boost the child's educational achievement.

Another important component of education is the neighbourhood where the child grows up. We denote this variable U . In the code we assume that the parent and child grow up in the same neighbourhood (have the same U effect). U is drawn from a standard normal distribution and the effect on education is given through a multiplication by the factor α .

In addition to this, we assume that there is a random component as well which differs between parent and child.

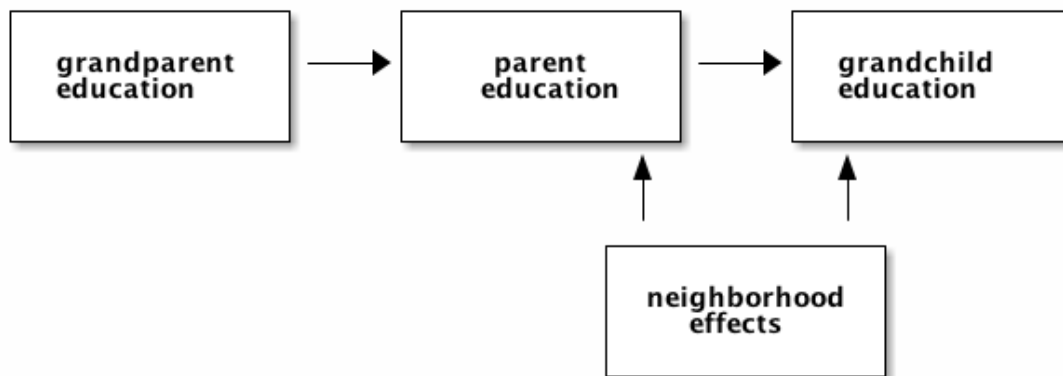


Figure 3: The education example from McElreath (2020).

The code below, generates this data and a dataframe `df_collider` that stores this data.

Look carefully at the code: is there a direct effect from the grandparent to the grandchild's educational achievement?

We create the dataframe `df_collider` which contains the observable variables X, Y, Z and the unobserved neighbourhood effect U .

```
import pandas as pd
import statsmodels.formula.api as smf
sample_size = 5000
alpha = 0.5
X = np.random.normal(size=sample_size)
U = np.random.normal(size=sample_size)
Z = X + alpha*U+0.4*np.random.normal(size=sample_size)
Y = Z + alpha*U+0.4*np.random.normal(size=sample_size)

df_collider = pd.DataFrame({'X':X, 'Y':Y, 'Z':Z, 'U':U})
print(df_collider.head())
```

Now we run the regression of Y on X controlling for Z .

```
results = smf.ols('Y ~ X + Z', data=df_collider).fit()
print(results.summary())
```

It turns out that grandparent education X has a negative effect on the grandchild's educational achievement Y . How is that possible?

To illustrate what is happening, we use an interactive plot made with altair.

To condition on Z (as we do in a multiple regression), we can drag vertically an interval for Z in the bottom histogram. This selection of Z turns blue in the scatter plot above and you can drag the selection up and down (conditioning on different values of Z).

Move this selection up and down and see what happens. In particular:

- for a selection of Z in the bottom histogram, what is the relation between X and Y in the scatter plot?
- for this selection, how does U vary in the figure? That is, for a given selection of Z what is the size of the circles? What is the interpretation? For given Z , as grandparent education X is low, what is the size of the U circles?

```
import altair as alt

interval = alt.selection_interval(encodings=['y'])

fig = (
    alt.Chart(df_collider)
    .mark_point()
    .encode(
        x='X',
        y='Y',
        color=alt.condition(
            interval,
            alt.value('steelblue'),
            alt.value('lightgreen')
        ),
        size='U'
    )
    .add_params(interval)
)

hist = (
    alt.Chart(df_collider)
    .mark_bar()
    .encode(
        x='count()',
        y=alt.Y('Z', bin=True),
    )
)
```

```
        color=alt.condition(
            interval,
            alt.value('steelblue'),
            alt.value('lightgrey')
        )
    )
    .add_params(interval)
)

chart_collider = fig & hist
chart_collider.save('./figures/ChartCollider.html')
```

collider bias with real data

Downey (2023, chap. 7) discusses a great example of collider bias suggesting (incorrectly) that mothers who smoke during pregnancy have a “protective effect” on their babies if these happen to have low birth weight.

- you can watch his presentation: probably overthinking low birthweight paradox
- or look at the notebook: <https://allendowney.github.io/ProbablyOverthinkingIt/birthweight.html>
- the website of the book: <https://greenteapress.com/wp/probably-overthinking-it/>

If you want to run this code on the Tilburg university server, you can access the data with:

```
from zipfile import ZipFile
zip_file = ZipFile('./data/lbw_data.csv.zip')
df = pd.read_csv(zip_file.open('lbw_data.csv'))
df.head()
```

Then follow Allen Downey’s presentation/notebook/book chapter.

Summary

This lecture introduced two practical pitfalls in variable selection and causal interpretation.

The multiple testing problem shows that significance is not a reliable guide to variable selection: if you test many variables, some will appear significant by chance, with an expected false-discovery rate of exactly significance level α .

The fork, pipe, and collider show that whether to control for a third variable Z depends entirely on the causal structure. Controlling for a fork variable is correct. Controlling for a pipe mediator or a collider is incorrect; it either blocks a causal path or introduces a spurious correlation that was not there before.

Looking ahead: In the next lecture we cover a number of “technical” Python topics that we need for later lectures on datascience methods.

Review Questions

Test your understanding. For each question, write the code in your own marimo notebook.

Question 1: False discovery rate

Repeat the multiple testing simulation but now generate k variables where exactly half have a true slope of 0.5 (and half are null).

a. What fraction of the true-effect variables do you correctly detect (power)? b. Among the “significant” results, what fraction are false discoveries? c. How does the false-discovery fraction change as you increase k while keeping the number of true signals fixed?

Question 2: Fork with partial effect

In the fork simulation above, X has no direct effect on Y . Now add a direct effect:

```
Y = 0.5 * X + gamma * Z + np.random.normal(0, 1, n)
```

a. What coefficient do you expect for X in the regression $Y \sim X$ (without controlling for Z)? b. What coefficient do you expect for X in the regression $Y \sim X + Z$? c. Run both regressions and check whether the simulated coefficients match your expectations.

TODO Question 3: Costly data acquisition

Suppose you work at a ministry and you and your team need to figure out what the effect is of a policy x on an outcome y . You can gather data on this relation between x and y but the problem is that the data acquisition is rather expensive. Hence, your colleagues suggest that you gather data sequentially (over time) until you find that the coefficient of x (on y) is significant.

Hopefully, in your statistics/econometrics classes you learned that this is a very bad idea. If not, I tell you: this is a bad idea.

Use data simulation to convince your colleagues that collecting data in this way is a bad idea.

TODO Question 4: Treatment effects

One of the complications when using IV to find causal effects is that treatment effects can differ between subgroups and that these subgroups react differently to the treatment/instrument. Our discussion here is based on Chapter 4 of Angrist and Pischke (2009).

We will consider this in the context of unemployed who are offered a training program. To understand the effects, we will again generate our own data such that we know exactly what all the relevant effects are. The question is then, how do we get these effects from the data.

As before, the learning goals of this section are two fold: (i) learn how to program your own data generating process in python and (ii) understand the statistical issue under consideration: heterogeneous treatment effects.

The government introduces a training program for unemployed. The treatment variable D_i denotes whether (1) or not (0) individual i received training. The instrument Z_i that we use is whether (1) or not (0) i is invited to join the training.

We have three groups of unemployed:

- compliers join the training if and only if they are invited to do so: $D_i = 1(0)$ iff $Z_i = 1(0)$;
- always-takers join the training whether or not they are invited to do so: $D_i = 1$ irrespective of Z_i ;
- never-takers never join the training: $D_i = 0$ irrespective of Z_i .

Note that in this training context, the always-takers are a bit “odd”: how can you join a training program where you are not supposed to be. The never-takers make more sense in this context: some people may not attend the training even though they were invited to do so. We will not worry about this; the point is that we can conceptually distinguish these groups.

We are interested in the effect of the training program on earnings. We denote by β the expected earnings without training. The expected earnings with training is denoted by $\beta + \tau$. Hence, τ denotes the treatment effects.

Let $j \in \{c, a, n\}$ denote whether an individual is a complier, always-taker or never-taker. We assume that β_j and τ_j vary with j .

In particular we assume that the earnings effect for someone in group j is drawn from a normal distribution with mean $\mu = \beta_j$ and standard deviation $\sigma = 1$ in case of no training and the effect of training is drawn from a normal distribution with mean $\mu = \tau_j$ and standard deviation $\sigma = 1$.

Below we will consider different configurations for β_j, τ_j and see how they affect the estimated effects.

Exercise Read the code below and explain what it does. E.g. can you explain how earnings depend on whether someone was invited for the training program?

```
“python def accept_complier(invited): return invited
def accept_always(invited): return np.ones_like(invited)
def accept_never(invited): return np.zeros_like(invited)

N_simulations = 10000 fraction_training = 0.2 # 20% of agents are invited to the training
types = ['complier','always_taker', 'never_taker']
β = {'complier': 1, 'always_taker': 2, 'never_taker': 1}
τ = {'complier': 3, 'always_taker': 4, 'never_taker': 1}
n = {'complier': 0.5, 'always_taker': 0.25, 'never_taker': 0.25}
accept = {'complier': accept_complier, 'always_taker': accept_always, 'never_taker': accept_never}
earnings_without = {}
training_effect = {}
earnings = {}
invited = {}
trained = {}
df = pd.DataFrame()
def data_simulation(β = β, τ=τ, n = n):
    for j in types:
        earnings_without[j] = np.random.normal(loc = β[j], scale = 1.0, size = int(n[j]*N_simulations))
        training_effect[j] = np.random.normal(loc = τ[j], scale = 1.0, size = int(n[j]*N_simulations))
        invited[j] = np.random.binomial(1,fraction_training,size = int(n[j]*N_simulations))
        trained[j] = accept[j](invited[j])
        earnings[j] = earnings_without[j]+ trained[j]*training_effect[j]
    column_invited = np.concatenate([invited[j] for j in types],axis=0)
    column_trained = np.concatenate([trained[j] for j in types],axis=0)
    column_earnings = np.concatenate([earnings[j] for j in types],axis=0)
    df = pd.DataFrame({'invited':column_invited, 'trained':column_trained,'earnings':column_earnings})
    return df

df = data_simulation() df.head() “
```

We have a dataset now with indicators for who was invited to a training and who attended a training.

We would like to learn the effect of training on earnings.

Question Which of the parameters above could we hope to recover? τ_c, τ_a, τ_n ?

Question Hence, if we do some calculations with the data, which number do we want to see?

Question Compare the average earnings of people with training with the average earnings of people without training. Is this the number that you expected? Why (not)?

Exercise Where does this number come from?

Question Perhaps we should use the instrument whether someone is invited or not: compare average earnings of people with and without an invitation to the training.

Question Does it help if we compare:

- people with training and an invitation to training **with**
- people without training and no invitation to training

We have seen now that we have no hope of recovering the relevant training effect in the set-up above. So the question is: when can we recover the relevant parameters? That is, what assumptions are needed?

To see how you can analyze this with the set-up above, let's program two obvious cases where this works.

Question Explain for each case **why** it works.

```
“python n = {'complier': 1.0, 'always_taker': 0.0, 'never_taker': 0.0} df_compliers_only =
data_simulation(n=n) np.mean(df_compliers_only[df_compliers_only.trained==1].earnings)-
np.mean(df_compliers_only[df_compliers_only.trained==0].earnings) “
```

```
“python β = {'complier': 1, 'always_taker': 1, 'never_taker': 1} τ = {'complier': 3, 'always_taker': 3,
'never_taker': 3} df_homog=data_simulation(β=β,τ=τ) np.mean(df_homog[df_homog.trained==1].earnings)-
np.mean(df_homog[df_homog.trained==0].earnings) “
```

For a less obvious case where we can retrieve relevant parameters, we will use Angrist and Pischke (2009: chapter 4). In particular, we will verify Theorem 4.4.2.

We work with one sided noncompliance. That is, we allow for never-takers but not for always-takers.

According to Theorem 4.4.2 the effect of treatment on the treated can be found as:

$$\frac{E(Y_i|Z_i = 1) - E(Y_i|Z_i = 0)}{Prob(D_i = 1|Z_i = 1)}$$

We first create the dataframe.

```
“python n={'complier': 0.5, 'always_taker': 0.0, 'never_taker': 0.5} df_one_sided = data_simulation(n=n)
“
```

Question Calculate $Prob(D_i = 1|Z_i = 1)$ and denote this ‘correction_term’.

Question Calculate $\frac{E(Y_i|Z_i=1)-E(Y_i|Z_i=0)}{Prob(D_i=1|Z_i=1)}$.

Question Which parameter have we recovered?

Probability of treatment

Consider a situation which is simpler than the one above in the sense that there are no always-takers, nor never-takers.

However, now the instrument (“invited”) Z causes the probability of training to increase from 0.2 to 0.6. 30% of people get invited.

Exercise Copy/paste python code from the analysis above to generate a dataframe using that $Z_i = 1$ implies that the probability of treatment increases from 0.2 to 0.6

Exercise Suppose we can observe whether someone received training or not. Use this to calculate the effect of training on earnings.

Suppose we cannot observe whether someone actually did the training or not (e.g. whether or not someone invested effort in the training). We only know (“from a previous study”) that explicitly inviting someone for the training (“nudge”) increases the probability that they invest training effort from 0.2 to 0.6.

Exercise Use the dataframe above to verify what the effect of the nudge is. Call this the ‘correction_term’. What value should the correction term have?

Angrist and Pischke (2009: chapter 4) Theorem 4.4.1 (LATE Theorem; local average treatment effect) claims that (under some assumptions), we can calculate the effect of training on earnings as:

$$\frac{E(Y_i|Z_i = 1) - E(Y_i|Z_i = 0)}{E(D_i|Z_i = 1) - E(D_i|Z_i = 0)}$$

Exercise Use this expression to calculate the effect of training on earnings.

Now we are going to use an IV estimation to find the same result. Run the following two regressions.

“python results_first_stage = smf.ols(‘trained ~ invited’, data=df_prob).fit() results_second_stage = smf.ols(‘earnings ~ invited’, data=df_prob).fit() “

Exercise Use these regressions to calculate the effect of training on earning.

Hint

- `np.cov(W, Y) [0, 1]` gives the sample covariance between W and Y .

- For the first stage in 2SLS: use the `ols` function from the Python Code Explanation section. The fitted values are $X_{\text{hat}} = b[0] + b[1] * W$.
 - The 2SLS estimator equals the IV estimator when there is exactly one instrument; they differ with multiple instruments.
-

Lecture 3: Tensors, broadcasting, indexing, matrix operations

Skills you learn in this lecture: working with multi-dimensional arrays (tensors), selecting data with slicing and fancy indexing, combining arrays with broadcasting, performing matrix algebra, and understanding gradient descent — the algorithm behind neural network training.

Learning Objectives

After completing this lecture you should be able to:

- use `.shape`, `.ndim`, and `.reshape` to inspect and transform numpy arrays of any dimension
- select subsets of arrays using slicing notation and boolean (fancy) indexing
- apply broadcasting rules to combine arrays of different shapes without explicit loops
- perform matrix operations with numpy: product `@`, transpose `.T`, inverse and rank via `numpy.linalg`
- implement gradient descent from scratch and explain how the learning rate controls convergence to the OLS solution

Introduction

In Lectures 1 and 2 we worked with one- and two-dimensional arrays: vectors of observations and matrices with rows as observations and columns as variables. Real-world data often has a more complicated structure than that. An image is a 2D grid of pixel intensities; a dataset of images adds a third dimension for the image index. A panel of incomes across countries, ages and time periods can be viewed as a 3D array. Machine learning frameworks handle this kind of data through *tensors*; numpy arrays generalized to any number of dimensions.

Three technical tools make tensor computations easy to handle; once you master them. *Slicing* and *indexing* let you extract subsets of a tensor in a single expression without loops. *Broadcasting* lets you combine arrays of different shapes by automatically stretching the smaller array to match the larger one; this replaces many nested loops with a single vectorized expression. *Matrix operations* are the language of linear algebra: the matrix product `@`, the transpose `.T`, and the inverse via `numpy.linalg` underpin OLS, regularized regression, and covariance matrices.

The second half of the lecture introduces *gradient descent*, the optimization algorithm inside TensorFlow, PyTorch, and every modern neural network library. We implement it from scratch for a simple linear regression: rather than solving the normal equations analytically, we minimize the mean squared error by iteratively nudging the slope and intercept in the direction that reduces the loss. The interactive app lets you watch this process in real time and experiment with the learning rate.

Apps

This lecture has two interactive apps. The first lets you enter two 3×3 matrices and observe matrix operations: addition, subtraction, multiplication, transposition, and inverse. The second app runs gradient descent for a small linear regression and shows both the loss surface and the convergence path.

Interactive matrix operations

Enter values for two 3×3 matrices A and B . The app instantly displays their sum, difference, matrix products AB and BA (where these two matrix products are usually not the same), the transpose A^T , the determinant and rank of A , and the inverse A^{-1} – provided A has full rank. Try setting one row of A to a linear combination of another row to see the inverse become undefined.

https://janboone.github.io/datascience-marimo/lecture3_matrices.html

Gradient descent for linear regression

The app generates 60 observations from a known linear relationship and asks gradient descent (starting at slope = 0, intercept = 0) to recover the best-fit line. The left panel shows the mean squared error $L(m, b)$ as a contour plot (representing a bowl in 3D) with the gradient-descent path overlaid. The OLS estimate sits at the bottom of the bowl (lowest point). The right panel tracks the loss at each iteration.

Use the sliders to vary the learning rate η and the number of steps. With small η the algorithm converges slowly; push η too high and the updates overshoot and the algorithm diverges.

https://janboone.github.io/datascience-marimo/lecture3_gradient_descent.html

Python Code Explanation

Open a `marimo` notebook alongside this document and type the code yourself. We build up from basic tensor manipulation to gradient descent.

Tensors and shape

A *tensor* is a numpy array with one or more dimensions. Every array has a `.shape` attribute – a tuple giving its size along each axis – and `.ndim` for the number of dimensions.

```
import numpy as np

x = np.random.normal(0, 1, 100)      # 1-D tensor: 100 numbers
print(x.shape, x.ndim)

M = np.random.normal(0, 1, (4, 5))   # 2-D tensor (a matrix)
print(M.shape, M.ndim)

T = np.random.normal(0, 1, (3, 28, 28)) # 3-D: 3 images of 28x28 pixels
print(T.shape, T.ndim)
```

Use `.reshape` to reorganize the elements without changing their values, as long as the total count stays the same.

```
x2 = x.reshape(10, 10)
x3 = x.reshape(4, 5, 5)
print(x2.shape)
print(x3.shape)
```

`np.newaxis` inserts a dimension of size 1; useful for broadcasting. The slicing notation `x[:5]` means: take the 5 first elements of `x` along axis zero.

```
col = x[:5, np.newaxis]
row = x[:5][np.newaxis, :]
print(col.shape, row.shape)
```

Instead of `np.newaxis` you can also use `None` like `x[:5, None]`.

Slicing and indexing

Slicing uses the notation `start:stop:step` for each dimension, separated by commas. `:` means “all elements”. The notation `::2` then means “every other element”.

```
x = np.arange(1, 101).reshape(10, 10) # 10x10 matrix, values 1 to 100

x[0, :]      # first row, all columns
x[:, 4]      # fifth column (index starts at 0)
x[:2, :2]    # top-left 2x2 block
```

```
x[0, 5:]          # second half of the first row
x[0, 5::2]        # every other element from that half
x[1::2, 1::2]     # every other row and column, starting from index 1
```

The last command yields:

```
array([[ 12,  14,  16,  18,  20],
       [ 32,  34,  36,  38,  40],
       [ 52,  54,  56,  58,  60],
       [ 72,  74,  76,  78,  80],
       [ 92,  94,  96,  98, 100]])
```

Fancy indexing selects elements using a boolean array.

```
x[x > 50]          # values exceeding 50, as a 1-D array
x[(x >= 30) & (x <= 40)] # & for "and", | for "or"
```

The last command gives:

```
array([30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40])
```

Boolean masks from one array can select from another of the same shape. The classic use: colour scatter-plot points by sign.

```
t = np.linspace(0, 2 * np.pi, 300)
y = np.sin(t)
import matplotlib.pyplot as plt
plt.plot(t[y >= 0], y[y >= 0], "b") # positive part: blue
plt.plot(t[y < 0], y[y < 0], "r") # negative part: red
```

Broadcasting

Broadcasting lets numpy combine arrays of different shapes by stretching dimensions of size 1 to match the other array. The broadcasting rule: check from the rightmost dimension leftward whether:

1. Dimensions of equal size: fine.

2. One dimension equals 1: fine
3. Neither of the above: error.

A practical example from panel econometrics: individual fixed effects I_i and time effects T_t , giving predictions $y_{it} = I_i + T_t$.

```
I = np.arange(0, 100, 10)    # 10 individual effects, shape (10,)
T = np.arange(3)             # 3 time effects, shape (3,)

y = I[:, np.newaxis] + T[np.newaxis, :]    # (10,1) + (1,3) → (10,3)
print(y.shape)                            # (10, 3)
print(y[9, 1])                           # 91 = 90 + 1 ✓
```

No nested loop over i and t — a single expression covers all 30 combinations.

Practice broadcasting yourself by defining two tensors:

```
x = np.random.normal(0,1,size=100)
y = np.random.normal(0,1,size=100)
```

and then asking which of the following can be added? First, think about it. Then check by running the code in a cell (see whether or not you get an error).

- `x+y`
- `x.reshape(10,10)+y`
- `x.reshape(10,10)+y.reshape(4,5,5)`
- `x.reshape(10,10)+y.reshape(10,10,1)`
- `x.reshape(10,10)+y.reshape(1,10,10)`
- `x.reshape(10,10)+y.reshape(5,10,2)`
- `x.reshape(10,10)+y.reshape(100,1,1)`

For more information on broadcasting check this [numpy documentation](#).

Matrix operations

The key matrix operations in numpy with k by l matrix A and m by n matrix B :

Expression	Meaning
$A @ B$	matrix product (must be the case that $l = m$)
$A * B$	element-wise product (shapes must broadcast)
$A.T$	transpose
<code>np.linalg.solve(A, b)</code>	solve $Ax = b$ (A must be a square matrix: $k = l$)
<code>np.linalg.inv(A)</code>	matrix inverse A^{-1} , must be square
<code>np.linalg.det(A)</code>	determinant
<code>np.linalg.matrix_rank(A)</code>	rank
<code>np.diag(v)</code>	diagonal matrix from vector, or extract diagonal

Matrix product `@` differs from element-wise multiplication `*`. By doing the calculations by hand, check the following:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])

print(A * B)
print("-----")
print(A @ B)
```

Before we try to calculate A^{-1} , let's check whether A has full rank:

```
print(np.linalg.det(A))
print(np.linalg.matrix_rank(A))
print(np.linalg.inv(A))
```

As $\det(A) \neq 0$, A has full rank; and, indeed, the rank of A equals 2 (which is max. rank for a 2 by 2 matrix). Hence, we can calculate the inverse of A .

We can check whether the inverse is correct:

```
np.linalg.inv(A) @ A
```

As we saw in the previous lecture, we can write OLS in matrix form as $\hat{\beta} = (X^T X)^{-1} X^T y$ and covariance matrix $\hat{\sigma}^2 (X^T X)^{-1}$.

We first simulate our own data: n observations and 4 x variables. We then define y ; what are the slope variable and what is the intercept?

```
n, k = 200, 4
X = np.random.normal(0, 1, (n, k))
y = X @ np.array([1, 0.5, -0.5, 0.3]) + np.random.normal(0, 1, n)
```

We define the matrix X with observations, derive our estimated coefficients $\hat{\beta}$, calculate the residuals which we need for the standard errors:

```
Xmat = np.column_stack([np.ones(n), X])
beta_hat = np.linalg.solve(Xmat.T @ Xmat, Xmat.T @ y)
resid = y - Xmat @ beta_hat
sigma2 = np.sum(resid ** 2) / (n - k - 1)
se = np.sqrt(np.diag(sigma2 * np.linalg.inv(Xmat.T @ Xmat)))
print("estimates: ", beta_hat.round(3))
print("std errors: ", se.round(3))
```

A square $n \times n$ matrix $A = X^\top X$ is invertible if and only if $\det(A) \neq 0$ or equivalently $\text{rank}(A) = n$. In statistics, a near-zero determinant indicates *multicollinearity*: one predictor is a linear combination of the others.

Gradient descent from scratch

The OLS formula $\hat{\beta} = (X^\top X)^{-1} X^\top y$ minimizes the MSE analytically. For models with no closed-form solution (like a neural network) we use *gradient descent*: start somewhere and step downhill.

For simple linear regression $\hat{y} = mx + b$ the MSE loss is

$$L(m, b) = \frac{1}{n} \sum_i (y_i - mx_i - b)^2$$

The gradients are

$$\frac{\partial L}{\partial m} = \frac{2}{n} \sum_i (mx_i + b - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (mx_i + b - y_i)$$

Update rule with learning rate η : $m \leftarrow m - \eta \partial L / \partial m$ and similarly for b . Here we set a seed (42) to make sure that your results will be the same as the ones below. We start our gradient descent at

$(m, b) = (0, 0)$ and use learning rate $\eta = 0.3$. Note that we use the derivatives of L derived above in our updating of m and b .

```
import numpy as np
import tensorflow as tf

rng = np.random.default_rng(42)
n = 60
x = np.sort(rng.uniform(0, 2, n))
y = 0.5 * x + 0.3 + rng.normal(0, 0.2, n)

# OLS:
Xmat = np.column_stack([np.ones(n), x])
b_hat = np.linalg.solve(Xmat.T @ Xmat, Xmat.T @ y)
print(f"OLS: intercept = {b_hat[0]:.4f}, slope = {b_hat[1]:.4f}")

# Gradient descent starting from (m, b) = (0, 0):
lr, m, b = 0.3, 0.0, 0.0
for step in range(200):
    resid = (m * x + b) - y
    m -= lr * 2 * np.mean(resid * x)
    b -= lr * 2 * np.mean(resid)

print(f"GD: intercept = {b:.4f}, slope = {m:.4f}")
```

In TensorFlow the same loop is written with `tf.Variable`, `tf.GradientTape` (which computes gradients automatically), and `assign_sub` (which does the subtraction update in-place). We will see this syntax when we build neural networks. For now, consider the following example which is copied from this SciPy 2019 Tutorial.

We define the tensorflow variables m, b and give them a starting value. Tensorflow now knows that m, b are variables that will be varied (trained) later on.

```
m = tf.Variable(0.)
b = tf.Variable(0.)
```

We can look at the variables, but they are not “easy to look at” as they contain more information than just the value. If we are only interested in the value, we can use the `numpy()` method on the variable.

```
print(m)
print(m.numpy())
```

We define a function that predicts y for given x . Note that `tf` “knows” that m, b are variables because of the way we defined these variables above. Hence, they are not explicitly mentioned in the function definition as variables.

```
def predict(x):
    y = m * x + b
    return y
```

We define our loss function: sum of squared differences between y_{pred} and y_{true} .

```
def squared_error(y_pred, y_true):
    return tf.reduce_mean(tf.square(y_pred - y_true))
```

To see what `reduce_mean` does, consider the following example. If the tensor vector has more than 1 dimension, you can specify the axis along which the reduction needs to take place.

```
vector = tf.constant([[1., 2., 3., 4.]])
print(tf.reduce_mean(vector).numpy())
```

As above, we use gradient descent to gradually improve our guess for m and b .

```
learning_rate = 0.3
steps = 200

for i in range(steps):

    with tf.GradientTape() as tape:
        predictions = predict(x)
        loss = squared_error(predictions, y)

    gradients = tape.gradient(loss, [m, b])

    m.assign_sub(gradients[0] * learning_rate)
    b.assign_sub(gradients[1] * learning_rate)

    if i % 20 == 0:
        print("Step %d, Loss %f" % (i, loss.numpy()))
```

Two things in the code above are new: `GradientTape` and `assign_sub`. We will discuss these briefly. More information on `GradientTape` can be found [here](#). The idea is that tensorflow calculates the gradient for us; we do not take the derivative of L w.r.t. m, b ourselves (as we did in the numpy code above).

As a simple example to get an idea of what it does, we work with the function z^2 . `gradient` calculates the derivative (it does so analytically) and fills in the value, in this case 3.0. Hence, the derivative equals $2 * z = 6.0$ at $z = 3.0$.

```
z = tf.Variable(3.0)
def my_function():
    return z**2

with tf.GradientTape() as tape:
    value = my_function()
tape.gradient(value, [z])
```

Finally, `assign_sub` subtracts the value from the variable. It is the equivalent of $z - 1.0$:

```
z = tf.Variable(3.0)
z.assign_sub(1.0)
```

Summary

This lecture introduced the technical Python toolkit needed for machine learning.

Tensors are numpy arrays with any number of dimensions; `.shape` reveals their structure, and `.reshape` reorganizes elements without copying data. *Slicing* and *fancy indexing* extract any subset in a single expression. *Broadcasting* eliminates loops when combining arrays of different shapes.

Matrix algebra underpins OLS and much of statistics: `@` for matrix products, `.T` for transposition, `np.linalg.solve` and `np.linalg.inv` for solving systems. A matrix is invertible if and only if it has full rank ($\det \neq 0$); near-zero determinants signal multicollinearity.

Gradient descent minimizes a loss function iteratively. The learning rate η controls the trade-off between speed and stability: too small means slow convergence; too large means divergence. This is the same algorithm used by TensorFlow and PyTorch to train neural networks.

Looking ahead: In the next lecture we study overfitting and underfitting and see why splitting data into train and test sets is essential whenever the model is flexible enough to memorize the training data.

Review Questions

Test your understanding. For each question, write the code in your own markdown notebook.

Question 1: Slicing and fancy indexing on panel data

Suppose you have simulated a small panel dataset: 500 observations on 8 variables. Variable 0 is a binary indicator (0/1), variable 3 is income, and variable 7 is an outcome of interest.

```
import numpy as np

rng = np.random.default_rng(0)
data = rng.normal(0, 1, (500, 8))
data[:, 0] = (rng.uniform(0, 1, 500) > 0.5).astype(float) # binary
```

Using only slicing and fancy indexing (no loops):

- a. Extract variables 3 and 7 for the first 100 observations. b. Select all rows where variable 0 equals 1 (the “treated” group). c. Among the treated group, compute the mean of variable 7 and compare it to the mean of variable 7 in the untreated group. d. Select all rows where variable 3 (income) is above its 75th percentile. Hint: `np.percentile`.
-

Question 2: Sampling distributions without loops

In Lecture 1 we plotted the sampling distribution of the mean by drawing many samples. Use broadcasting to simulate $R = 10,000$ draws of the sample mean for *each* of the sample sizes $n \in \{10, 50, 100, 500\}$ simultaneously — with no Python loop over either R or n .

- a. Generate a matrix of shape $(R, n_{\max})$ of i.i.d. draws from $\mathcal{N}(0, 1)$ where $n_{\max} = 500$. Then use slicing to obtain the sample mean for each n and each repetition. Hint: for sample size n , the sample mean of repetition r is `data[r, :n].mean()`. Can you compute all of these without a loop? b. For each n , plot the histogram of the R sample means and verify that the spread decreases as n grows. c. Verify numerically that the standard deviation of the sample mean is approximately $1/\sqrt{n}$.
-

Question 3: Multicollinearity and the matrix inverse

Start with the OLS example from the Python Code Explanation section ($n=200$, $k=4$ predictors). Then add a fifth predictor that is *exactly* the sum of predictors 1 and 2.

```
X_extra = np.column_stack([X, X[:, 0] + X[:, 1]]) # 5th column is linear
↳ combo
Xmat_extra = np.column_stack([np.ones(n), X_extra])
```

a. Compute `np.linalg.det(Xmat_extra.T @ Xmat_extra)` and `np.linalg.matrix_rank(Xmat_extra.T @ Xmat_extra)`. What do you observe? b. Try to call `np.linalg.solve(Xmat_extra.T @ Xmat_extra, Xmat_extra.T @ y)`. What happens? c. Add a tiny amount of noise to the fifth column: `X_extra[:, 4] + 1e-6 * rng.normal(size=n)`. Recompute the determinant and solve. Does OLS now run? What do the estimated coefficients look like?

This illustrates why near-multicollinearity is dangerous even when `solve` does not error: the estimates become unstable.

Question 4: Batched OLS with `np.matmul`

In this question you run N OLS regressions simultaneously — one per dataset — using numpy tensor operations on the last two dimensions, with no Python loop.

Generate $N = 500$ datasets, each with $n = 100$ observations and $k = 3$ predictors (plus intercept). The true slope vector for all datasets is $\beta = (1, 0.5, -0.5, 0.3)^\top$ (intercept first).

```
N, n, k = 500, 100, 3
rng = np.random.default_rng(7)
X_all = rng.normal(0, 1, (N, n, k)) # shape (N, n, k)
y_all = X_all @ np.array([0.5, -0.5, 0.3]) + 1.0 + rng.normal(0, 1, (N, n))
```

a. Add an intercept column to `X_all` so that it has shape $(N, n, k+1)$. Hint: `np.ones((N, n, 1))` and `np.concatenate`. b. Compute $X^\top X$ of shape $(N, k+1, k+1)$ using `np.matmul` (or `@`) on the last two dimensions. Remember that “ X transpose times X ” requires transposing the *last two* dimensions of `X_all`. Use `np.swapaxes(X_all, -1, -2)` to do this without rearranging the batch dimension. c. Similarly compute $X^\top y$ of shape $(N, k+1)$ or $(N, k+1, 1)$. d. Solve all N systems $(X^\top X)\hat{\beta} = X^\top y$ in one call using `np.linalg.solve`. The result `beta_hat` should have shape $(N, k+1)$. Verify that `beta_hat.mean(axis=0)` is close to the true coefficients.

Hint: how to prompt an LLM

If you get stuck on parts (b)–(d), use the following prompt (adapted to what you have already tried):

I have a numpy tensor `X_all` of shape (N, n, p) (N datasets, n observations each, p predictors) and a tensor `y_all` of shape (N, n) . I want to compute the OLS coefficient vector $\hat{\beta} = (X^T X)^{-1} X^T y$ for all N datasets simultaneously using `np.matmul` or `@`, without any Python loop. The result should have shape (N, p) . How do I transpose only the last two dimensions of `X_all`, form $X^T X$ and $X^T y$, and then solve the N linear systems at once?

After you get a working solution, check that it gives the same answer as a loop:

```
beta_loop = np.array([
    np.linalg.solve(X_all[i].T @ X_all[i], X_all[i].T @ y_all[i])
    for i in range(N)
])
print(np.allclose(beta_hat, beta_loop))    # should print True
```

Lecture 4: Overfitting and underfitting

Skills you learn in this lecture: splitting data into train and test sets, diagnosing over-fitting and under-fitting by comparing train and test errors, regularizing with ridge and lasso regression, and tuning the regularization strength with cross-validation.

Learning Objectives

After completing this lecture you should be able to:

- explain why adding more parameters always improves the training fit but may worsen out-of-sample predictions
- create a polynomial design matrix with `np.vander` and fit it by OLS with `np.linalg.solve`
- implement a train/test split with numpy slicing and compute the mean squared error on each set
- implement ridge regression using the closed-form solution $\hat{\beta}_{\text{ridge}} = (X^T X/n + \lambda \Lambda)^{-1} X^T y/n$
- implement lasso regression and explain why it produces exact zeros for coefficients
- tune the hyper-parameter λ with k fold cross-validation

Introduction

In machine learning and econometrics we face a fundamental tension: a more complex model (more parameters) always fits the training data better, but a model that is *too* complex will fit the noise in the training sample rather than the underlying signal. That is, the model picks up structure that is specific for the sample and does not generalize to the population.

Adding more explanatory variables to a regression cannot make the training fit worse. The OLS minimization problem,

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i(\beta))^2$$

can only improve (or stay the same) when β is extended. If adding the new variable hurt, we would simply set its coefficient to zero and recover the previous fit.

The problem is that a model optimized on the training sample can pick up idiosyncratic patterns — noise — that do not repeat in new data. We say the model does not generalize. Two failure modes matter:

- *Under-fitting*: the model is too simple to capture the true relationship. Both training and test errors are high.
- *Over-fitting*: the model is too complex and fits the training noise. Training error is low but test error is high.

The standard diagnostic is a *train/test split*: hold out a random subset of the data, train the model on the remainder, and evaluate predictions on the held-out test set. The test error is an unbiased estimate of generalization performance. Crucially, the test set should be used *once*, after we have settled on a final model. Repeatedly comparing models on the test set risks over-fitting to it.

A practical remedy for over-fitting is *regularization*: add a penalty for large coefficients to the objective function. Two popular penalties are the *ridge* (L_2) penalty $\lambda \sum_j \beta_j^2$ and the *lasso* (L_1) penalty $\lambda \sum_j |\beta_j|$. As λ increases, coefficients shrink toward zero; with a very large λ even complex models under-fit. The penalty strength λ is a *hyper-parameter* chosen by cross-validation, not by fitting/estimating on the training set.

Apps

This lecture has two interactive apps. The first shows how train and test errors diverge as polynomial degree increases. The second applies ridge and lasso regularization to an over-parameterized polynomial and illustrates how the optimal penalty balances the bias–variance trade-off.

Two remarks; first, we are not particularly interested in polynomial regression. But it is an intuitive way in which to make the model more complex and the over-fitting pattern is obvious in a simple figure. Second, in the app the train/test split of the data is done on sorted data which is not advisable in practice but helps to illustrate the effect in the figures.

Polynomial regression: over-fitting and under-fitting

Data come from the true model $y = 1 + 2x^2 + \varepsilon$. We split 30 observations into 20 training and 10 test points and fit a polynomial of degree d only to the training data.

Use the slider to vary d from 1 to 9. The top figure shows all data together with the fitted curve; the two panels below compare the fit on the training set (left) and on the test set (right). The train MSE falls monotonically with d ; the test MSE is minimized near the true degree 2 and grows thereafter.

https://janboone.github.io/datascience-marimo/lecture4_overfitting.html

Ridge and lasso regularisation

We fix a degree-9 polynomial (10 coefficients, only 20 training observations) and add ridge and lasso regularization. Use the slider to vary λ .

- Train MSE rises as λ increases (more shrinkage = more bias).
- Test MSE first falls (over-fitting reduced) and then rises (under-fitting).
- Lasso sets some coefficients exactly to zero; ridge keeps all non-zero but small.

https://janboone.github.io/datascience-marimo/lecture4_regularization.html

Python Code Explanation

Open a marimo notebook alongside this document and type the code yourself. We build up from polynomial features and the train/test split to regularized regression and cross-validation.

Polynomial features and train/test split

We generate data from the true quadratic model and split into train and test sets using numpy slicing. We split the data on sorted data for “dramatic effect” in our figures.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
N, n_train = 30, 20
x = np.sort(rng.uniform(-2.0, 2.0, N))
y = 1.0 + 2.0 * x ** 2 + rng.normal(0, 1.5, N)

x_train, x_test = x[:n_train], x[n_train:]
y_train, y_test = y[:n_train], y[n_train:]
```

A *polynomial design matrix* of degree d has columns $[1, x, x^2, \dots, x^d]$. `np.vander` builds this efficiently with the `increasing=True` argument.

```
def poly_features(xv, degree):
    return np.vander(xv, N=degree + 1, increasing=True)

d = 4
Xtr = poly_features(x_train, d)  # shape (20, 5)
Xte = poly_features(x_test, d)  # shape (10, 5)
Xtr.shape                       # (20, 5)
```

We fit OLS on the training data and compute the mean squared error (MSE) on both sets.

```
def ols(X, y):
    return np.linalg.solve(X.T @ X, X.T @ y)

beta = ols(Xtr, y_train)
mse_train = np.mean((y_train - Xtr @ beta) ** 2)
mse_test = np.mean((y_test - Xte @ beta) ** 2)
print(f"Train MSE: {mse_train:.2f}, Test MSE: {mse_test:.2f}")
```

To see how MSE changes with polynomial degree we loop over degrees 1 to 9.

```
degrees = range(1, 10)
train_errors, test_errors = [], []
for d in degrees:
    Xtr_d = poly_features(x_train, d)
    Xte_d = poly_features(x_test, d)
    b = ols(Xtr_d, y_train)
```

```

train_errors.append(np.mean((y_train - Xtr_d @ b) ** 2))
test_errors.append(np.mean((y_test - Xte_d @ b) ** 2))

plt.plot(degrees, train_errors, label="Train MSE")
plt.plot(degrees, test_errors, label="Test MSE")
plt.axvline(2, color="gray", linestyle="--", label="True degree (d=2)")
plt.xlabel("Polynomial degree"); plt.ylabel("MSE")
plt.yscale("log"); plt.legend()

```

The train MSE curve is monotonically decreasing; the test MSE curve has a U-shape with its minimum near the true degree 2.

Ridge regression: shrinking coefficients

Ridge regression adds a squared-coefficient penalty to the OLS objective:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \frac{1}{n} \|y - X\beta\|^2 + \lambda \sum_{j=1}^p \beta_j^2$$

The first-order condition gives the closed-form solution

$$\hat{\beta}_{\text{ridge}} = \left(\frac{X^\top X}{n} + \lambda \Lambda \right)^{-1} \frac{X^\top y}{n}$$

where $\Lambda = \text{diag}(0, 1, 1, \dots, 1)$ so the intercept is not penalized. With $\lambda = 0$ this coincides with the OLS estimate.

Before fitting, we standardize the polynomial features (not the intercept column) so that λ treats all predictors equally regardless of their natural scale.

```

# Standardise columns 1..d using training mean/std
def standardize(Xraw, mean=None, std=None):
    if mean is None:
        mean = Xraw[:, 1:].mean(axis=0)
        std = Xraw[:, 1:].std(axis=0) + 1e-8
    Xs = Xraw.copy()
    Xs[:, 1:] = (Xs[:, 1:] - mean) / std
    return Xs, mean, std

d = 9

```

```

Xtr_raw = poly_features(x_train, d)
Xte_raw = poly_features(x_test, d)

Xtr, fmean, fstd = standardize(Xtr_raw)
Xte, _, _ = standardize(Xte_raw, fmean, fstd)

def ridge(X, y, lam):
    n, p = X.shape
    Lambda = np.diag([0.0] + [1.0] * (p - 1))
    A = X.T @ X / n + lam * Lambda
    return np.linalg.solve(A, X.T @ y / n)

for lam in [0.0, 0.5, 1.0, 2.0]:
    b = ridge(Xtr, y_train, lam)
    mse_tr = np.mean((y_train - Xtr @ b) ** 2)
    mse_te = np.mean((y_test - Xte @ b) ** 2)
    print(f"lambda={lam:.1f}  train MSE={mse_tr:.2f}  test MSE={mse_te:.2f}")

```

Ridge shrinks all coefficients continuously but never sets any exactly to zero.

Lasso regression: coordinate descent and sparsity

The lasso uses an L_1 penalty:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \sum_{j=1}^p |\beta_j|$$

There is no closed-form solution. The single-coordinate minimizer involves the *soft-thresholding* operator

$$S(z, \lambda) = \text{sign}(z) \cdot \max(|z| - \lambda, 0)$$

which sets the coefficient to zero whenever $|z| < \lambda$.

Starting from the coordinate-wise objective:

$$\min_{\beta_j} \frac{1}{2n} \|r_j - X_j \beta_j\|^2 + \lambda |\beta_j|$$

with $r_j = y - \sum_{k \neq j} X_k \beta_k$

Expanding the quadratic term:

$$\frac{1}{2n} (r_j^T r_j - 2\beta_j X_j^T r_j + \beta_j^2 X_j^T X_j) + \lambda |\beta_j|$$

Taking the derivative of the smooth part:

$$\frac{\partial}{\partial \beta_j} = \frac{1}{n} (-X_j^T r_j + \beta_j X_j^T X_j)$$

The first order condition for the full objective is:

$$\frac{1}{n} (-X_j^T r_j + \beta_j X_j^T X_j) + \lambda \partial |\beta_j| = 0$$

where the notation $\partial |\beta_j|$ is short-hand for the following “derivative”:

$$\partial |\beta_j| = \begin{cases} \{1\} & \beta_j > 0 \\ \{-1\} & \beta_j < 0 \\ [-1, 1] & \beta_j = 0 \end{cases}$$

Letting $z_j = X_j^T r_j / n$ and $c_j = X_j^T X_j / n$, the three cases resolve to:

$$\hat{\beta}_j = \begin{cases} \frac{z_j - \lambda}{c_j} & z_j > \lambda \\ \frac{z_j + \lambda}{c_j} & z_j < -\lambda \\ 0 & |z_j| \leq \lambda \end{cases}$$

Which is exactly the soft-threshold formula:

$$\hat{\beta}_j = \frac{S(z_j, \lambda)}{c_j}$$

```
def soft_threshold(z, threshold):
    return np.sign(z) * np.maximum(np.abs(z) - threshold, 0.0)

def lasso_cd(X, y, lam, max_iter=3000, tol=1e-8):
    n, p = X.shape
    beta = np.zeros(p)
    col_norms = np.sum(X ** 2, axis=0) / n  # yields the c_j
    for _ in range(max_iter):
```

```

    beta_old = beta.copy()
    for j in range(p):
        # Partial residual holding all other coefficients fixed
        r_j = y - X @ beta + X[:, j] * beta[j]
        z_j = X[:, j] @ r_j / n
        if j == 0:                                # intercept: no penalty
            beta[j] = z_j / col_norms[j]
        else:
            beta[j] = soft_threshold(z_j, lam) / col_norms[j]
    if np.max(np.abs(beta - beta_old)) < tol:
        break
    return beta

for lam in [0.0, 0.5, 1.0, 2.0]:
    b = lasso_cd(Xtr, y_train, lam)
    n_zeros = np.sum(np.abs(b[1:]) < 1e-6)
    mse_te = np.mean((y_test - Xte @ b) ** 2)
    print(f"lambda={lam:.1f}  zeros={n_zeros}/9  test MSE={mse_te:.2f}")

```

Lasso sets coefficients exactly to zero (sparse solution) because the L_1 penalty has a kink at zero: any coefficient smaller than λ in magnitude is thresholded all the way to zero. This makes lasso a natural tool for variable selection. Ridge, which uses a smooth L_2 penalty, shrinks coefficients (in absolute value) but does not set them equal to zero.

Tuning λ with cross-validation

The hyper-parameter λ controls the bias-variance trade-off. We choose it by k fold cross-validation: split the training data into k equal folds; in each iteration use $k - 1$ folds to fit the model and the remaining fold to evaluate the loss. The average validation loss across folds estimates out-of-sample performance without ever touching the test set.

When you will use k fold cross validation in your own analyses, you will probably use scikit-learn KFold. Here we program the k folds ourselves with numpy. Read the code for the function `kfold_cv` and try to understand how this works (you can ask an LLM for help, if necessary).

```

def kfold_cv(X, y, lam_values, k=5, method="ridge"):
    n = len(y)
    fold_size = n // k # integer division
    val_mse = np.zeros(len(lam_values))
    for fold in range(k):

```

```

idx_val = np.arange(fold * fold_size, (fold + 1) * fold_size)
idx_train = np.concatenate([np.arange(0, fold * fold_size),
                             np.arange((fold + 1) * fold_size, n)])
Xf_tr, yf_tr = X[idx_train], y[idx_train]
Xf_val, yf_val = X[idx_val], y[idx_val]
for i, lam in enumerate(lam_values):
    if method == "ridge":
        b = ridge(Xf_tr, yf_tr, lam)
    else:
        b = lasso_cd(Xf_tr, yf_tr, lam)
    val_mse[i] += np.mean((yf_val - Xf_val @ b) ** 2)
return val_mse / k

lam_values = np.linspace(0.05, 3.0, 61)
cv_ridge = kfold_cv(Xtr, y_train, lam_values, k=5, method="ridge")
cv_lasso = kfold_cv(Xtr, y_train, lam_values, k=5, method="lasso")

opt_ridge = lam_values[np.argmin(cv_ridge)]
opt_lasso = lam_values[np.argmin(cv_lasso)]

plt.plot(lam_values, cv_ridge, label="Ridge CV MSE")
plt.plot(lam_values, cv_lasso, label="Lasso CV MSE")
plt.axvline(opt_ridge, linestyle="--", color="steelblue", alpha=0.6,
            ↪ label="optimal  $\lambda$  ridge")
plt.axvline(opt_lasso, linestyle="--", color="forestgreen", alpha=0.6,
            ↪ label="optimal  $\lambda$  lasso")
plt.xlabel("lambda"); plt.ylabel("Validation MSE")
plt.legend()

```

Both curves have a U-shape: too small λ leads to over-fitting; too large leads to under-fitting. The optimal λ sits at the bottom of the U. Because we only have $k = 5$ folds on 20 training points (4 per fold), the validation estimates are noisy; with more data the U becomes smoother and the optimum sharper.

Bias-variance decomposition

Assume the true relationship is $y = f(x) + \varepsilon$, where $\varepsilon \sim (0, \sigma^2)$ is independent noise. Let $\hat{y}$ be our estimator. We want to compute the expected squared error at a fixed point x :

$$\begin{aligned}
\mathbb{E}[(y - \hat{y})^2] &= \mathbb{E}[(f(x) + \varepsilon - \hat{y})^2] \\
&= \mathbb{E}[(f(x) - \hat{y} + \varepsilon)^2] \\
&= \mathbb{E}[(f(x) - \hat{y})^2] + 2 \mathbb{E}[(f(x) - \hat{y}) \varepsilon] + \mathbb{E}[\varepsilon^2].
\end{aligned}$$

Since ε is independent of $\hat{y}$ and $\mathbb{E}[\varepsilon] = 0$, the cross term vanishes:

$$\mathbb{E}[(f(x) - \hat{y}) \varepsilon] = \mathbb{E}[f(x) - \hat{y}] \mathbb{E}[\varepsilon] = 0,$$

and $\mathbb{E}[\varepsilon^2] = \sigma^2$. So:

$$\mathbb{E}[(y - \hat{y})^2] = \mathbb{E}[(f(x) - \hat{y})^2] + \sigma^2.$$

Now we decompose the first term. Let $\mu = \mathbb{E}[\hat{y}]$. Add and subtract μ :

$$\begin{aligned}
\mathbb{E}[(f(x) - \hat{y})^2] &= \mathbb{E}[(f(x) - \mu + \mu - \hat{y})^2] \\
&= \mathbb{E}[(f(x) - \mu)^2 + 2(f(x) - \mu)(\mu - \hat{y}) + (\mu - \hat{y})^2] \\
&= (f(x) - \mu)^2 + 2(f(x) - \mu) \mathbb{E}[\mu - \hat{y}] + \mathbb{E}[(\hat{y} - \mu)^2].
\end{aligned}$$

Since $\mathbb{E}[\mu - \hat{y}] = \mu - \mu = 0$, the cross term vanishes, leaving:

$$\mathbb{E}[(f(x) - \hat{y})^2] = \underbrace{(f(x) - \mu)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{y} - \mu)^2]}_{\text{Var}(\hat{y})}.$$

Combining everything the expected test error decomposes into three terms:

$$\mathbb{E}[(y - \hat{y})^2] = \underbrace{(\mathbb{E}[\hat{y}] - f(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{y})}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

- *Bias*: how far the average prediction is from the truth. A too-simple model (underfitting) has high bias.
- *Variance*: how much the prediction changes across different training samples. A too-complex model (overfitting) has high variance.
- *Irreducible noise*: σ^2 , the noise in y itself. No model can remove it.

Regularisation trades bias for variance: as λ increases, bias rises (the model is forced toward zero) but variance falls (the model is less sensitive to the particular training sample). The optimal λ balances the two.

We can verify this decomposition empirically by repeating the experiment many times with different training samples and observing the average squared bias and variance of the predictions.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

N_sim = 500
x_grid = np.linspace(-1.0, 1.0, 80)
degrees = [1, 5, 9]

for ax, d in zip(axes, degrees):
    all_preds = np.zeros((N_sim, len(x_grid)))
    for i in range(N_sim):
        xi = np.sort(rng.uniform(-2.0, 2.0, n_train))
        yi = 1.0 + 2.0 * xi ** 2 + rng.normal(0, 1.5, n_train)
        coeffs_i = np.polyfit(xi, yi, d)
        all_preds[i] = np.polyval(coeffs_i, x_grid)

    f_true = 1.0 + 2.0 * x_grid ** 2
    mean_pred = all_preds.mean(axis=0)
    bias_sq = (mean_pred - f_true) ** 2
    variance = all_preds.var(axis=0)

    ax.plot(x_grid, bias_sq, label="Bias2")
    ax.plot(x_grid, variance, label="Variance")
    ax.plot(x_grid, bias_sq + variance, label="Bias2 + Variance", lw=2)
    ax.set_xlabel("x")
    ax.set_ylabel("MSE component")
    ax.set_title(f"Degree-{d} polynomial")
    ax.legend()

fig.suptitle("Bias-Variance Decomposition", fontsize=14)
plt.tight_layout()
```

Compare the total Bias² + Variance across degrees 1, 5 and 9 to trace the full bias–variance trade-off curve. At degree 1 bias is large; at degree 9 variance dominates.

Why lasso gives sparsity but ridge does not

Both penalties shrink coefficients toward zero, but only lasso can set them *exactly* to zero. The reason is geometric.

Consider minimising the OLS loss subject to a budget constraint on the coefficients:

- *Ridge* constrains $\|\beta\|_2^2 = \beta_1^2 + \beta_2^2 \leq c$. The constraint region is a smooth circle (sphere in higher dimensions). The loss contours (ellipses) typically touch the constraint region on its curved surface, away from the axes, so both coefficients are non-zero.
- *Lasso* constrains $\|\beta\|_1 = |\beta_1| + |\beta_2| \leq c$. The constraint region is a diamond with corners on the axes. The loss contours are much more likely to first touch the diamond at a corner, where one coefficient is exactly zero.

We can visualise this in 2D.

```
theta = np.linspace(0, 2 * np.pi, 300)
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))

# OLS contours (loss ellipses centred away from origin)
beta_ols_2d = np.array([1.2, 0.8]) # hypothetical OLS solution
A2 = np.array([[2.0, 0.5], [0.5, 1.0]]) # positive definite quadratic form

b1, b2 = np.meshgrid(np.linspace(-1.5, 2.0, 200), np.linspace(-1.5, 1.5,
    ↪ 200))
beta_grid = np.stack([b1, b2], axis=-1)
diff = beta_grid - beta_ols_2d
loss_2d = np.einsum("...i,ij,...j->...", diff, A2, diff)

for ax, title, constraint in zip(axes, ["Ridge (L2)", "Lasso (L1)"], ["L2",
    ↪ "L1"]):
    ax.contour(b1, b2, loss_2d, levels=10, cmap="Blues_r", alpha=0.7)
    if constraint == "L2":
        ax.fill(np.cos(theta), np.sin(theta), alpha=0.3, color="steelblue")
        ax.plot(np.cos(theta), np.sin(theta), color="steelblue", lw=2)
    else:
        diamond = np.array([[1,0],[0,1],[-1,0],[0,-1],[1,0]])
        ax.fill(diamond[:,0], diamond[:,1], alpha=0.3, color="forestgreen")
        ax.plot(diamond[:,0], diamond[:,1], color="forestgreen", lw=2)
    ax.axhline(0, color="gray", lw=0.8); ax.axvline(0, color="gray", lw=0.8)
    ax.set_xlabel(r"$\beta_1$"); ax.set_ylabel(r"$\beta_2$")
    ax.set_title(title); ax.set_xlim(-1.6, 2.1); ax.set_ylim(-1.6, 1.6)

plt.tight_layout()
```

The corner of the lasso diamond lies on the $\beta_2 = 0$ axis: wherever the OLS solution points from that corner, the unconstrained minimizer must cross the horizontal axis, making it likely that the constrained solution lands exactly there with $\beta_2 = 0$.

Regularisation path

The *regularisation path* shows how each coefficient changes as λ varies from zero (OLS) to a large value (strong shrinkage). It helps identify which variables are most important: lasso paths hit zero at different λ values, effectively ordering variables by their relevance.

```
lam_path = np.logspace(-2, 1, 60) # from 0.01 to 10 on a log scale
ridge_coefs = np.array([ridge(Xtr, y_train, l)[1:] for l in lam_path])
lasso_coefs = np.array([lasso_cd(Xtr, y_train, l)[1:] for l in lam_path])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

for j in range(ridge_coefs.shape[1]):
    ax1.plot(lam_path, ridge_coefs[:, j])
ax1.set_xscale("log"); ax1.axhline(0, color="k", lw=0.8, ls="--")
ax1.set_xlabel("lambda (log scale)"); ax1.set_ylabel("Coefficient value")
ax1.set_title("Ridge regularisation path (all coefficients shrink smoothly)")

for j in range(lasso_coefs.shape[1]):
    ax2.plot(lam_path, lasso_coefs[:, j])
ax2.set_xscale("log"); ax2.axhline(0, color="k", lw=0.8, ls="--")
ax2.set_xlabel("lambda (log scale)"); ax2.set_ylabel("Coefficient value")
ax2.set_title("Lasso regularisation path (coefficients hit zero at different lambda)")

plt.tight_layout()
```

In the ridge path all curves approach zero asymptotically but never reach it. In the lasso path curves bend sharply and hit zero at specific λ values. The last variable to reach zero is the most important predictor according to lasso.

Information criteria

An alternative to cross-validation that you may find in the literature is an *information criterion*: penalize the log-likelihood (or equivalently the in-sample fit) for model complexity directly, without holding out data. Two common choices are:

- AIC: $AIC = n \log(RSS/n) + 2k$
- BIC: $BIC = n \log(RSS/n) + k \log n$

where k is the number of parameters and RSS is the residual sum of squares. BIC penalizes model size more heavily (by $\log n$ instead of 2) and tends to select simpler models.

Information criteria are fast to compute (no refitting needed) but assume the model is correctly specified. Cross-validation makes fewer assumptions and is preferred when data are plentiful.

```
def aic(y_train, y_pred_train, k):
    n = len(y_train)
    rss = np.sum((y_train - y_pred_train) ** 2)
    return n * np.log(rss / n) + 2 * k

def bic(y_train, y_pred_train, k):
    n = len(y_train)
    rss = np.sum((y_train - y_pred_train) ** 2)
    return n * np.log(rss / n) + k * np.log(n)

degrees = range(1, 10)
aic_vals, bic_vals = [], []
for d in degrees:
    Xtr_d = poly_features(x_train, d)
    b = ols(Xtr_d, y_train)
    k = d + 1  # number of free parameters
    aic_vals.append(aic(y_train, Xtr_d @ b, k))
    bic_vals.append(bic(y_train, Xtr_d @ b, k))

opt_aic = list(degrees)[np.argmin(aic_vals)]
opt_bic = list(degrees)[np.argmin(bic_vals)]
print(f"AIC selects degree {opt_aic}, BIC selects degree {opt_bic}")

plt.plot(degrees, aic_vals, "o-", label=f"AIC (min at d={opt_aic})")
plt.plot(degrees, bic_vals, "s-", label=f"BIC (min at d={opt_bic})")
plt.axvline(2, color="gray", ls="--", label="True degree")
plt.xlabel("Polynomial degree"); plt.ylabel("Information criterion")
plt.legend()
```

Both criteria should select a degree close to the true value of 2. BIC will generally prefer an equal or lower degree than AIC because its penalty grows with n .

Summary

- *Over-fitting* occurs when a model is so complex that it fits training-sample noise rather than the true data-generating process. Adding polynomial terms always reduces training MSE but eventually increases test MSE.
- A *train/test split* reveals over-fitting: if test MSE is much larger than train MSE the model has over-fit. The test set should be used only once to evaluate the final model.
- *Ridge regression* adds a squared-coefficient penalty $\lambda \|\beta\|_2^2$ to the OLS objective. Its closed-form solution $\hat{\beta} = (X^\top X/n + \lambda \Lambda)^{-1} X^\top y/n$ shrinks all coefficients toward zero without setting any exactly to zero.
- *Lasso regression* adds an absolute-value penalty $\lambda \|\beta\|_1$ which can be solved by coordinate descent. The *soft-thresholding* operator sets small coefficients exactly to zero, performing automatic variable selection. A useful rule of thumb: lasso selects at most as many variables as there are observations.
- The hyper-parameter λ is chosen by k fold cross-validation, which estimates out-of-sample performance by rotating which fold is held out for validation.

Review Questions

Lecture 5: Neural network

To get a first sense of how a neural network works, we look at two simple python programs that implement these algorithms. For this, we use the website from the book machine learning: an algorithmic perspective.

Below you find the code for the files pcn_logic_eg.py and mlp.py; which we adapted for python 3 by adjusting the print statements (should be `print()` in python 3).

By looking at the underlying code we get an idea how neural networks work. Then we create a neural network using tensorflow 2.0.

The tensorflow syntax is very similar to Keras. There is a datacamp course on deep learning with Keras and a course on tensorflow 2.0.

- define the relu and softmax functions and plot these functions

```
def relu(x):  
    return np.clip(x,0,np.inf)  
def softmax(x):  
    return [np.exp(x[i])/np.sum(np.exp(x)) for i in range(len(x))]
```

```
range_x = np.arange(10)-5
fig, ax = plt.subplots(1,2,figsize=(12,5))
ax[0].plot(range_x,relu(range_x))
ax[1].plot(range_x,softmax(range_x),'ro')
ax[0].set_title('relu')
ax[1].set_title('softmax');
```

```
np.sum(softmax(range_x))
```

why relu?

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Target: cubic polynomial
# -----
def f(x):
    return x**3 + x**2 - x - 1

def relu(z):
    return np.maximum(0.0, z)

# -----
# Build design matrix for fixed knots and slopes
# Model:  $\hat{y} = c_0 + \sum_{j=1..k} a_j * \text{relu}(s_j * (x - t_j))$ 
# -----
def design_matrix(x, knots, slopes):
    Phi = [np.ones_like(x)]
    for t, s in zip(knots, slopes):
        Phi.append(relu(s * (x - t)))
    return np.column_stack(Phi)

def fit_amplitudes(x, y, knots, slopes):
    Phi = design_matrix(x, knots, slopes)
    beta, *_ = np.linalg.lstsq(Phi, y, rcond=None)
    yhat = Phi @ beta
    mse = np.mean((y - yhat) ** 2)
```

```

    return beta, yhat, mse

# -----
# Stagewise: add one knot at a time, search slope for the new knot
# -----
def stagewise_fixed_knots_opt_slopes(x, y, knots, slope_grid):
    chosen_slopes = []
    history = []

    for k in range(1, len(knots) + 1):
        t_new = knots[k - 1]

        best = None
        for s_new in slope_grid:
            trial_slopes = chosen_slopes + [s_new]
            beta, yhat, mse = fit_amplitudes(x, y, knots[:k], trial_slopes)

            if (best is None) or (mse < best["mse"]):
                best = {"s_new": s_new, "beta": beta, "yhat": yhat, "mse":
↪ mse}

            chosen_slopes.append(best["s_new"])
            history.append({
                "k": k,
                "knots": knots[:k].copy(),
                "slopes": chosen_slopes.copy(),
                "beta": best["beta"],
                "yhat": best["yhat"],
                "mse": best["mse"],
            })

    return history

# -----
# Demo + plotting (2x3 subfigures)
# -----
# data
x_min, x_max = -2.0, 1.5
x = np.linspace(x_min, x_max, 500)
y = f(x)

# fixed knots (6)

```

```

knots = np.linspace(x_min, x_max, 6)

# small slope grid (keep it simple; widen if needed)
slope_grid = np.array([-8, -4, -2, -1, -0.5, 0.5, 1, 2, 4, 8], dtype=float)

hist = stagewise_fixed_knots_opt_slopes(x, y, knots, slope_grid)

# plot progression: 6 panels via subfigures(2,3)
fig = plt.figure(figsize=(12, 7), constrained_layout=True)
subfigs = fig.subfigures(2, 3)

for i, stage in enumerate(hist):
    r, c = divmod(i, 3)
    sf = subfigs[r, c]
    ax = sf.subplots(1, 1)

    ax.plot(x, y, label="target")
    ax.plot(x, stage["yhat"], label="ReLU approx")

    # show knots used so far
    for t in stage["knots"]:
        ax.axvline(t, linewidth=0.8, alpha=0.3)

    ax.set_title(f"Stage {stage['k']} (MSE={stage['mse']:.2e})")
    ax.set_xlim(x_min, x_max)

    if i == 0:
        ax.legend(loc="best")

```

Perceptron

Next we copy from Chapter 3 of Machine Learning: An Algorithmic Perspective, the “perceptron”. The code is in the next cell. Try to understand the python code.

The main part of the algorithm is in the function (method, actually, but do not worry about this) `pc_ntrain`; the line

```
self.weights -= eta*np.dot(np.transpose(inputs),self.activations-targets)=
```

determines how the weights in the network are updated. This optimization step is discussed in this chapter of the datacamp course.

```
# Code from Chapter 3 of Machine Learning: An Algorithmic Perspective (2nd
↳ Edition)
# by Stephen Marsland (http://stephenmonika.net)

# You are free to use, change, or redistribute the code in any way you wish
↳ for
# non-commercial purposes, but please maintain the name of the original
↳ author.
# This code comes with no warranty of any kind.

# Stephen Marsland, 2008, 2014
```

```
class pcn:
    """ A basic Perceptron (the same pcn.py except with the weights printed
    and it does not reorder the inputs) """

    def __init__(self, inputs, targets):
        """ Constructor """
        # Set up network size
        if np.ndim(inputs)>1:
            self.nIn = np.shape(inputs)[1]
        else:
            self.nIn = 1

        if np.ndim(targets)>1:
            self.nOut = np.shape(targets)[1]
        else:
            self.nOut = 1

        self.nData = np.shape(inputs)[0]

        # Initialise network
        self.weights = np.random.rand(self.nIn+1, self.nOut)*0.1-0.05

    def pcntrain(self, inputs, targets, eta, nIterations):
        """ Train the thing """
        # Add the inputs that match the bias node
        inputs = np.concatenate((inputs, -np.ones((self.nData, 1))), axis=1)

        # Training
```

```

change = range(self.nData)

for n in range(nIterations):

    self.activations = self.pcnfwd(inputs);
    self.weights -=
        ↪ eta*np.dot(np.transpose(inputs),self.activations-targets)
    print("Iteration: ", n)
    print(self.weights)

    activations = self.pcnfwd(inputs)
    print("Final outputs are:")
    print(activations)
    #return self.weights

def pcnfwd(self,inputs):
    """ Run the network forward """

    # Compute activations
    activations = np.dot(inputs,self.weights)

    # Threshold the activations
    return np.where(activations>0,1,0)

def confmat(self,inputs,targets):
    """Confusion matrix"""

    # Add the inputs that match the bias node
    inputs = np.concatenate((inputs,-np.ones((self.nData,1))),axis=1)
    outputs = np.dot(inputs,self.weights)

    nClasses = np.shape(targets)[1]

    if nClasses==1:
        nClasses = 2
        outputs = np.where(outputs>0,1,0)
    else:
        # 1-of-N encoding
        outputs = np.argmax(outputs,1)
        targets = np.argmax(targets,1)

    cm = np.zeros((nClasses,nClasses))

```

```

    for i in range(nClasses):
        for j in range(nClasses):
            cm[i,j] =
↪ np.sum(np.where(outputs==i,1,0)*np.where(targets==j,1,0))

    print(cm)
    print(np.trace(cm)/np.sum(cm))

```

Below you see 4 data points with a target value (0 or 1) for each data point. We need to predict the target value. We plot this with a red color for target 0 and blue for target 1.

Predicting the target here means finding a straight line such that all red points are on one side of the line and the blue points on the other side of the line. Since this example is very simple, you can draw the line yourself (I hope...). This simplicity helps us to understand what the algorithm does.

We define a tensor with 4 inputs and 4 targets. The first element of inputs (which has two coordinates) corresponds to the first element in targets.

To refer to the points in the figure, we use “standard” terminology x and y .

```

inputs = np.array([[0,0],[0,1],[1,0],[1,1]])
targets = np.array([[0],[1],[1],[1]])
colors = []
for i in range(len(targets)):
    colors.append(['red','blue'][targets[i][0]])
plt.scatter(inputs[:,0],inputs[:,1],color=colors)
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.show()

```

Now we use the pcn class and instantiate it with the inputs and targets above. We call p an instance of the class pcn. In this course we will not dwell on the object oriented aspects of python, so do not worry if you do not fully understand this. However, as you will see below, when we use tensorflow, we have a similar structure: we create an instance of a tensorflow class model.

The function definition is pcntrain(self, inputs, targets, eta, nIterations). Do not worry about the self part. We call the function as pcntrain(inputs, targets, 0.25, 6). Hence, the first two arguments of the function are our inputs and targets tensors.

The result is that p is now associated with our tensors inputs and targets. Now we can use the function (“method”) pcntrain to train the network. After each iteration, the algorithm prints some information. Looking at pcntrain above, we can see that the information that is printed is:

```
print("Iteration: ", n)
print(self.weights)
print("Final outputs are:")
print(activations)
```

So first we see the number of the iteration (and python starts counting at 0). Then we see three numbers that correspond to the weights. The weights determine the line in the figure above. In particular, if we write the weights tensor as $w = (w_0, w_1, w_2)$, then a line in the figure is given by $w_0x + w_1y = w_2$. Equivalently, we can write this as:

$$y = (w_2 - w_0x)/w_1 \quad (1)$$

Next, we see the final outputs (targets): this is the “prediction” of the algorithm for the vector `targets`. Recall that `targets` is of the form `[0,1,1,1]`. Hence, after the first iteration, we incorrectly label the first point as 1 while it should be 0. Hence, the algorithm continues and generates different (and ultimately better) predictions.

```
p = pcn(inputs,targets)
p.pcntrain(inputs,targets,0.25,6)
```

The final weights w are now equal to:

```
p.weights
```

We can now plot the line $y = (w_2 - w_0x)/w_1$. Indeed, as you can see, the line separates the red point (below the line) from the blue points (above the line). If we give the algorithm a point above the line, it will predict “blue”, if we give it a point below the line it will predict “red”.

Because we only have 4 points here, this is arbitrary for points close to the line: we can shift the line, still separate the points, but get different predictions for points close to the line.

```
x0 = 0
x1 = 1
y0 = (p.weights[2]-p.weights[0]*x0)/p.weights[1]
y1 = (p.weights[2]-p.weights[0]*x1)/p.weights[1]
plt.scatter(inputs[:,0],inputs[:,1],color=colors)
plt.plot([x0,x1],[y0,y1],'k') #we only need to give two points to plot a line
plt.xlabel('$x$')
```

```
plt.xlabel('$y$')
plt.show()
```

We can see from the graph and check with the function `pcnfwd` below what the predicted labels will be for the points (0.5,0.5) and (0.5,-0.5). The -1 is needed for the “constant” w_2 .

```
p.pcnfwd([[0.5,0.5,-1],[0.5,-0.5,-1]])
```

To get a “graphical” intuition what a neural network does, see this website and play around with the dataset that you want to use, the number of hidden layers, number of neurons per hidden layer and the features that you want to use.

E.g. choose the dataset where the orange points “circle” the blue points. Use only 1 hidden layer (linear model) and 2 neurons in this layer. Then you cannot perfectly separate the points. Play around with the options to see how you can separate the points.

simple example

From the python code for a neural network:

```
self.weights -= eta*np.dot(np.transpose(inputs),self.activations-targets)
```

In equation format, the weights are updated as: $w = w - \eta(\text{prediction} - \text{target})x$

Let’s look at a simple example to understand why this the correct updating rule. We have 4 observations with two labels (targets): red and blue in the figure.

We want to find a line such that all points above the line are blue; the one point below the line is red.

```
inputs = np.array([[0,0],[0,1],[1,0],[1,1]])
targets = np.array([[0],[1],[1],[1]])
colors = []
for i in range(len(targets)):
    colors.append(['red','blue'][targets[i][0]])
plt.scatter(inputs[:,0],inputs[:,1],color=colors)
plt.plot([0,1],[0.8,0.2],c='k')
plt.plot([0,1],[0.8,-0.2], '--',c='k')

plt.xlabel('$x$')
plt.ylabel('$y$')
plt.show()
```

Line of the form $w_0x + w_1y - w_2 = 0$ or $y = -w_0/w_1x + w_2/w_1$.

The solid line mis-classifies one point. To get this classification right, the line needs to become steeper for the point with $x = 1$ and $y = 0$.

To make the line steeper, w_0 needs to increase:

$$w_0 = w_0 - \eta(0 - 1) * 1 = w_0 + \eta \quad (2)$$

If we give the model a new point, it will predict “blue” if the point is above the line and “red” if it is below.

Multi-layer perceptron

With more than one layer, we get into deep learning. This is what “deep” in deep learning refers to: more than one layer.

We use this mlp code from Chapter 4 of Machine Learning: An Algorithmic Perspective.

```
# Code from Chapter 4 of Machine Learning: An Algorithmic Perspective (2nd
↪ Edition)
# by Stephen Marsland (http://stephenmonika.net)

# You are free to use, change, or redistribute the code in any way you wish
↪ for
# non-commercial purposes, but please maintain the name of the original
↪ author.
# This code comes with no warranty of any kind.

# Stephen Marsland, 2008, 2014

import numpy as np

class mlp:
    """ A Multi-Layer Perceptron """

    def
    ↪ __init__(self, inputs, targets, nhidden, beta=1, momentum=0.9, outtype='logistic'):
        """ Constructor """
        # Set up network size
        self.nin = np.shape(inputs)[1]
```

```

self.nout = np.shape(targets)[1]
self.ndata = np.shape(inputs)[0]
self.nhidden = nhidden

self.beta = beta
self.momentum = momentum
self.outtype = outtype

# Initialise network
self.weights1 = (np.random.rand(self.nin+1,self.nhidden)-
    ↪ 0.5)*2/np.sqrt(self.nin)
self.weights2 = (np.random.rand(self.nhidden+1,self.nout)-
    ↪ 0.5)*2/np.sqrt(self.nhidden)

def earllystop-
    ↪ ping(self,inputs,targets,valid,validtargets,eta,niterations=100):

    valid =
    ↪ np.concatenate((valid,-np.ones((np.shape(valid)[0],1))),axis=1)

    old_val_error1 = 100002
    old_val_error2 = 100001
    new_val_error = 100000

    count = 0
    while (((old_val_error1 - new_val_error) > 0.001) or
    ↪ ((old_val_error2 - old_val_error1)>0.001)):
        count+=1
        print(count)
        self.mlptrain(inputs,targets,eta,niterations)
        old_val_error2 = old_val_error1
        old_val_error1 = new_val_error
        validout = self.mlpfwd(valid)
        new_val_error = 0.5*np.sum((validtargets-validout)**2)

    print("Stopped", new_val_error,old_val_error1, old_val_error2)
    return new_val_error

def mlptrain(self,inputs,targets,eta,niterations):
    """ Train the thing """
    # Add the inputs that match the bias node
    inputs = np.concatenate((inputs,-np.ones((self.ndata,1))),axis=1)

```

```

change = range(self.ndata)

updatew1 = np.zeros((np.shape(self.weights1)))
updatew2 = np.zeros((np.shape(self.weights2)))

for n in range(niterations):

    self.outputs = self.mlpfwd(inputs)

    error = 0.5*np.sum((self.outputs-targets)**2)
    if (np.mod(n,100)==0):
        print("Iteration: ",n, " Error: ",error)

    # Different types of output neurons
    if self.outtype == 'linear':
        deltao = (self.outputs-targets)/self.ndata
    elif self.outtype == 'logistic':
        deltao =
↪ self.beta*(self.outputs-targets)*self.outputs*(1.0-self.outputs)
    elif self.outtype == 'softmax':
        deltao = (self.outputs-targets)*(self.outputs*(-
↪ self.outputs)+self.outputs)/self.ndata
↪
    else:
        print("error")

    deltah = self.hidden*self.beta*(1.0-
↪ self.hidden)*(np.dot(deltao,np.transpose(self.weights2)))

    updatew1 = eta*(np.dot(np.transpose(inputs),deltah[:, :-1])) +
↪ self.momentum*updatew1
    updatew2 = eta*(np.dot(np.transpose(self.hidden),deltao)) +
↪ self.momentum*updatew2
    self.weights1 -= updatew1
    self.weights2 -= updatew2

    # Randomise order of inputs (not necessary for matrix-based
    ↪ calculation)
    #np.random.shuffle(change)
    #inputs = inputs[change,:]
    #targets = targets[change,:]

```

```

def mlpfwd(self,inputs):
    """ Run the network forward """

    self.hidden = np.dot(inputs,self.weights1);
    self.hidden = 1.0/(1.0+np.exp(-self.beta*self.hidden))
    self.hidden = np.concatenate((self.hidden,-
        ↪ np.ones((np.shape(inputs)[0],1))),axis=1)

    outputs = np.dot(self.hidden,self.weights2);

    # Different types of output neurons
    if self.outtype == 'linear':
        return outputs
    elif self.outtype == 'logistic':
        return 1.0/(1.0+np.exp(-self.beta*outputs))
    elif self.outtype == 'softmax':
        normalisers =
        ↪ np.sum(np.exp(outputs),axis=1)*np.ones((1,np.shape(outputs)[0]))
        return np.transpose(np.transpose(np.exp(outputs))/normalisers)
    else:
        print("error")

def confmat(self,inputs,targets):
    """Confusion matrix"""

    # Add the inputs that match the bias node
    inputs =
    ↪ np.concatenate((inputs,-np.ones((np.shape(inputs)[0],1))),axis=1)
    outputs = self.mlpfwd(inputs)

    nclasses = np.shape(targets)[1]

    if nclasses==1:
        nclasses = 2
        outputs = np.where(outputs>0.5,1,0)
    else:
        # 1-of-N encoding
        outputs = np.argmax(outputs,1)
        targets = np.argmax(targets,1)

    cm = np.zeros((nclasses,nclasses))
    for i in range(nclasses):

```

```
        for j in range(nclasses):
            cm[i,j] =
↪ np.sum(np.where(outputs==i,1,0)*np.where(targets==j,1,0))

    print("Confusion matrix is:")
    print(cm)
    print("Percentage Correct: ",np.trace(cm)/np.sum(cm)*100)
```

This code is more elaborate than the perceptron above. We will go over some of the concepts below.

Below we use the famous iris dataset. Admittedly, flowers are not directly linked to economics, but this is one of the classic datasets in classification. You cannot claim to have done “datascience” without having looked at the iris dataset.

We will first download it from the website, just to learn the relevant steps here. We will work with it in numpy. In fact, the data is so famous that it is included in the tensorflow library, but later you may want to analyze data that is not included in any library. Hence, we need to learn the download steps as well.

When you look at the downloaded data (in an editor), you will notice that it actually includes the names as strings. For us it is easier to work with numerical labels 0,1,2. Hence, we use the function `preprocessIris`. This function takes two arguments: the input file with the data that we downloaded and the output file where the target-labels are replaced by 0,1,2.

If you run this notebook for the first time, un-comment the line `preprocessIris('iris.data','iris_proc.data')` (that is, remove the hashtag “#” at the beginning of the line), where `iris.data` is the name of the file that you downloaded and `iris_proc.data` is the name of the file with labels 0,1,2. Once the file `iris_proc.data` is created, there is no need to run this python command again.

```
# Code from Chapter 4 of Machine Learning: An Algorithmic Perspective (2nd
↪ Edition)
# by Stephen Marsland (http://stephenmonika.net)

# You are free to use, change, or redistribute the code in any way you wish
↪ for
# non-commercial purposes, but please maintain the name of the original
↪ author.
# This code comes with no warranty of any kind.

# Stephen Marsland, 2008, 2014

# The iris classification example
```

```

def preprocessIris(infile,outfile):

    stext1 = 'Iris-setosa'
    stext2 = 'Iris-versicolor'
    stext3 = 'Iris-virginica'
    rtext1 = '0'
    rtext2 = '1'
    rtext3 = '2'

    fid = open(infile,"r")
    oid = open(outfile,"w")

    for s in fid:
        if s.find(stext1)>-1:
            oid.write(s.replace(stext1, rtext1))
        elif s.find(stext2)>-1:
            oid.write(s.replace(stext2, rtext2))
        elif s.find(stext3)>-1:
            oid.write(s.replace(stext3, rtext3))
    fid.close()
    oid.close()

## Preprocessor to remove the test (only needed once)
# preprocessIris('./data/iris.data','./data/iris_proc.data')

iris = np.loadtxt('./data/iris_proc.data',delimiter=',')

# we normalize the data (the features, not the labels):
iris[:,4] = iris[:,4]-iris[:,4].mean(axis=0)
imax =
    ↪ np.concatenate((iris.max(axis=0)*np.ones((1,5)),np.abs(iris.min(axis=0))*np.ones((1,5)))
iris[:,4] = iris[:,4]/imax[:,4]
print(iris[0:5,:])

# instead of using labels 0,1,2; we use labels like [1,0,0], [0,1,0],
    ↪ [0,0,1]; this is called one-hot encoding
# algorithm that you use determines what the labels should look like
# one-hot encoding is often used when analyzing text-data
target = np.zeros((np.shape(iris)[0],3));

```

```
indices = np.where(iris[:,4]==0)
target[indices,0] = 1
indices = np.where(iris[:,4]==1)
target[indices,1] = 1
indices = np.where(iris[:,4]==2)
target[indices,2] = 1

# Randomly order the data
order = np.arange(np.shape(iris)[0])
np.random.shuffle(order)
iris = iris[order,:]
target = target[order,:]

# Split into training, validation, and test sets
train = iris[:,0:4]
traint = target[:,2]
valid = iris[1::4,0:4]
validt = target[1::4]
test = iris[3::4,0:4]
testt = target[3::4]
```

Uncomment and evaluate the following cell if you want to get an idea what the data look like.

```
#print(iris)
#print(train.max(axis=0), train.min(axis=0))
```

Exercise Plot the data with one feature on the horizontal axis and another feature on the vertical axis. Then color the observation based on the label.

```
# YOUR CODE HERE
raise NotImplementedError()
```

Next we train the network:

```
# Train the network
#import mlp
net = mlp(train,traint,5,outtype='logistic')
net.earlystopping(train,traint,valid,validt,0.1)
net.confmat(test,testt)
```

As the choice of train, evaluate and test data is random, the results can differ here.

The idea of the confusion matrix is as follows: * the rows are the true labels, the columns are the predicted labels; * if all non-diagonal entries equal 0, the model gives a perfect prediction; * if the element on the 3 rd row and 1st column is positive: some observations that actually have the 3rd label are incorrectly predicted to have the 1st label.

When there are a lot of labels, looking at the confusion matrix is no longer helpful. A number of summary measures have been defined for such cases, like accuracy, sensitivity, precision etc.

neural network graphical illustration

neural network figure

Neural network with tensorflow

using history to check for over/under fitting

- return to neural network with the handwritten digits data set
- use `history` method to fine tune the number of epochs to avoid under/over-fitting

```
(train_images, train_labels), (test_images, test_labels) = \
    keras.datasets.mnist.load_data()

model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(train_images,
                    train_labels,
                    epochs=60,
                    batch_size=512,
                    validation_data=(test_images, test_labels))
```

```
loss = history.history['loss'][1:]
val_loss = history.history['val_loss'][1:]

epochs = range(1, len(loss)+1)

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

TODO experiment with architecture: number of nodes and layers; e.g. try to over- or under-fit

example neural network: confusion matrix

do this with marimo in molab

- download the data from the jupyterlab server
- rename the file iris.csv
- go to <https://molab.marimo.io/notebooks>
- sign in (e.g. with your github account)
- click the button “Generate with AI”
- upload the dataset iris.csv and use a prompt like: “Attached is the well known iris data set. Present two plots to illustrate how we want to separate the different iris-es. Split the data into train and test set. Estimate a neural network with tensorflow and show the history so that we can tune the number of epochs. Finally present a confusion matrix to judge how well the fit is.”
- https://molab.marimo.io/notebooks/nb_8cFqKF4gmDTrk2QAJ5B888
- get some intuition in the playground: <https://playground.tensorflow.org/>

my own code

We import some data:

```
import requests
url = 'https://archive.ics.uci.edu/ml/machine-learning-
↳ databases/iris/iris.data'
```

```
r = requests.get(url, allow_redirects=True)
filename = "./data/iris.csv"
open(filename, 'wb').write(r.content)

df = pd.read_csv(filename, header=None,
    ↪ names=['sepal_length', 'sepal_width', \
            'petal_length', 'petal_width', 'species'])
df.head()
```

```
df.shape
```

```
df.dtypes
```

```
df.species.unique()
```

```
df["species"].replace({"Iris-setosa":0, "Iris-virginica":1, \
    "Iris-versicolor":2}, inplace=True)
df.tail()
```

```
plt.scatter(df['sepal_width'], df['petal_width'], c=df.species);
```

```
plt.scatter(df['sepal_length'], df['petal_length'], c=df.species);
```

- Neural networks do not only use linear functions to separate the data
- <https://playground.tensorflow.org/>

Before estimating a neural network, it is a good idea to standardize the features. No need to standardize the labels.

```
X = np.zeros([150, 4])
i = 0
for col in df.columns[0:4]:
    x = df[col].values
    X[:, i] = (x - x.mean()) / x.std()
    i += 1

y = df['species'].values
```

- Ignoring for now splitting the data into test and train set

```
model = keras.Sequential([
    keras.layers.Dense(20, input_dim = 4, activation='relu'), # input layer
    keras.layers.Dense(10, activation = 'relu'),              # hidden
    keras.layers.Dense(3, activation = 'softmax')              # output
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(X,y,epochs=10)
```

```
y_pred = model.predict(X)
matrix = tf.math.confusion_matrix(y, y_pred.argmax(axis=1))
matrix
```

```
array([[49,  0,  1],
       [ 0, 36, 14],
       [ 0,  3, 47]], dtype=int32)>
```

```
tf.math.confusion_matrix?
```

```
print(y[-5:])
print(y_pred[-5:,:])
```

Economic Neural Networks

Nonlinear Phillips curve

We estimate a **nonlinear Phillips curve** for European countries using a small neural network:

- Target: monthly **HICP inflation (YoY)**.
- Features: **unemployment rate** and **lags** of unemployment and inflation.
- Data: **Eurostat** (HICP: prc_hicp_manr; Unemployment: une_rt_m)

- next week we see how we can download data from Eurostat

The goal is to show an **economic** application of neural nets: letting the network learn curvature/state-dependence in the Phillips relationship, rather than a fixed linear slope.

```
import pandas as pd
df = pd.read_csv('./data/data_phillips_curve.csv')
df.head()
```

- `urate` denotes unemployment rate
- `hicp_yoy` Harmonised Index of Consumer Prices (HICP) inflation is a standardized way to measure inflation across the EU; year-over-year (YoY)

```
# Choose features & target
X_cols = ["urate", "urate_lag1", "urate_lag2", "urate_lag3",
          "hicp_yoy_lag1", "hicp_yoy_lag2", "hicp_yoy_lag3"]
y_col = "hicp_yoy"

# Time split (e.g., train up to end-2018; validate out-of-sample afterward)
df["date"] = [pd.Timestamp(d) for d in df["date"].values]
cutoff_date = pd.Timestamp("2019-01-01")
train = df[df["date"] < cutoff_date].copy()
valid = df[df["date"] >= cutoff_date].copy()

# Standardize features by train stats
mu, sd = train[X_cols].mean(0), train[X_cols].std(0).replace(0, 1.0)
train_X, valid_X = (train[X_cols] - mu) / sd, (valid[X_cols] - mu) / sd
train_y, valid_y = train[y_col].values.astype("float32"), \
                  valid[y_col].values.astype("float32")

train_X = train_X.values.astype("float32")
valid_X = valid_X.values.astype("float32")

train.shape, valid.shape, train_X.shape, train_y.shape
```

We use a compact MLP with tanh (hyperbole tangent function) activations (instead of relu). Students can change widths, epochs, etc.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
tf.random.set_seed(0)

model = keras.Sequential([
    layers.Input(shape=(len(X_cols),)),
    layers.Dense(32, activation="tanh"),
    layers.Dense(16, activation="tanh"),
    layers.Dense(1)
])

model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse",\
              metrics=["mae"])

history = model.fit(
    train_X, train_y,
    validation_data=(valid_X, valid_y),
    epochs=60,
    batch_size=256,
    verbose=1
)

hist = pd.DataFrame(history.history)
#hist.tail()
```

```
import matplotlib.pyplot as plt

plt.figure()
plt.plot(hist["loss"], label="train MSE")
plt.plot(hist["val_loss"], label="valid MSE")
plt.xlabel("epoch"); plt.ylabel("loss")
plt.legend()
plt.title("Learning curves (MSE)")
plt.show()
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

pred_valid = model.predict(valid_X, verbose=0).squeeze()
mae = mean_absolute_error(valid_y, pred_valid)
mse = mean_squared_error(valid_y, pred_valid)

print(f"Validation MAE: {mae:.3f} | MSE: {mse:.3f}")
```

```

# plt.figure()
# plt.scatter(valid_y, pred_valid, s=10)
# plt.xlabel("Actual inflation (YoY)")
# plt.ylabel("Predicted inflation (YoY)")
# plt.title("Predicted vs Actual (validation)");

# plot a few countries as time series
countries_to_plot = valid["geo"].unique()[:3]
n = len(countries_to_plot)

fig, axes = plt.subplots(1, n, figsize=(5*n, 4), sharey=True)
if n == 1: # when there's only one country
    axes = [axes]

for ax, g in zip(axes, countries_to_plot):
    mask = (valid["geo"] == g).values
    sub = valid.loc[mask]
    yhat = pred_valid[mask] # align predictions with the same rows

    ax.plot(sub["date"], sub["hicp_yoy"], label="Actual")
    ax.plot(sub["date"], yhat, label="Predicted")
    ax.set_title(f"{g}: Actual vs Predicted")
    ax.set_xlabel("Date")
    ax.tick_params(axis="x", rotation=45)

axes[0].set_ylabel("YoY inflation")
axes[0].legend(loc="upper left")
plt.tight_layout()

```

Solving for equilibrium

- <https://cournot-nn.streamlit.app/>

Model

- Inverse demand: $p(x_1, x_2) = 1 - x_1 - x_2$.

- Costs: $C_i(x_i) = \frac{1}{2}c_i x_i^2$.
- Profits: $\pi_i = p(x_1, x_2)x_i - \frac{1}{2}c_i x_i^2$.

Closed-form solution (for verification)

Solving

$$(2 + c_1)x_1 + x_2 = 1, \quad x_1 + (2 + c_2)x_2 = 1$$

yields

$$x_1^*(c_1, c_2) = \frac{1 + c_2}{(2 + c_1)(2 + c_2) - 1}, \quad x_2^*(c_1, c_2) = \frac{1 + c_1}{(2 + c_1)(2 + c_2) - 1}.$$

Goal

Solve a Cournot duopoly by training a tiny neural network that maps cost parameters (c_1, c_2) to outputs (x_1, x_2) so that each firm's **first-order condition (FOC)** holds:

$$\frac{\partial \pi_i}{\partial x_i} = 0.$$

We keep the code as simple and readable as possible (no decorators, default TF dtypes).

```
import tensorflow as tf

def price(x1, x2):
    return 1.0 - x1 - x2

def closed_form_x(c1, c2):
    denom = (2.0 + c1) * (2.0 + c2) - 1.0
    x1_star = (1.0 + c2) / denom
    x2_star = (1.0 + c1) / denom
    return x1_star, x2_star

c1_min, c1_max = 0.0, 3.0
c2_min, c2_max = 0.0, 3.0

def sample_costs(n):
    c1 = tf.random.uniform((n,1), c1_min, c1_max)
    c2 = tf.random.uniform((n,1), c2_min, c2_max)
    return tf.concat([c1, c2], axis=1) # shape (n,2)

net = tf.keras.Sequential([
```

```

    tf.keras.layers.Dense(64, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(64, activation='tanh'),
    tf.keras.layers.Dense(2) # raw outputs -> we'll make them nonnegative
    ↪ below
])

opt = tf.keras.optimizers.Adam(1e-3)

def train_step(cc):
    c1, c2 = cc[:,0], cc[:,1]
    with tf.GradientTape() as outer:
        with tf.GradientTape(persistent=True) as inner:
            raw = net(cc, training=True)
            x1 = tf.nn.softplus(raw[:, 0])
            x2 = tf.nn.softplus(raw[:, 1])

            inner.watch([x1, x2])
            p = price(x1, x2)
            C1 = 0.5 * c1 * tf.square(x1)
            C2 = 0.5 * c2 * tf.square(x2)
            pi1 = p * x1 - C1
            pi2 = p * x2 - C2

            r1 = inner.gradient(pi1, x1)
            r2 = inner.gradient(pi2, x2)
        del inner

        loss = tf.reduce_mean(tf.square(r1)) + tf.reduce_mean(tf.square(r2))
        loss += 1e-2 * tf.reduce_mean(tf.square(tf.nn.relu(-p))) # penalty
        ↪ term if p < 0

        grads = outer.gradient(loss, net.trainable_variables)
        opt.apply_gradients(zip(grads, net.trainable_variables))
    return loss, x1, x2

batch_size = 512
epochs = 1500

for it in range(1, epochs+1):
    cc = sample_costs(batch_size)
    L, x1_hat, x2_hat = train_step(cc)

```

```

if it % 300 == 0:
    cc_test = sample_costs(256)
    c1t, c2t = cc_test[:, 0], cc_test[:, 1]
    raw_test = net(cc_test, training=False)
    x1_test = tf.nn.softplus(raw_test[:, 0])
    x2_test = tf.nn.softplus(raw_test[:, 1])
    x1_star, x2_star = closed_form_x(c1t, c2t)
    mae = tf.reduce_mean(tf.abs(x1_test - x1_star)) + \
        tf.reduce_mean(tf.abs(x2_test - x2_star))
    print(f"iter {it:4d} | loss {L.numpy():.3e} | test MAE
          ↪ {mae.numpy():.3e}")

c1_ex = tf.constant([[1.0]])
c2_ex = tf.constant([[1.5]])
raw_ex = net(tf.concat([c1_ex, c2_ex], axis=1), training=False)
x1_ex = tf.nn.softplus(raw_ex[:, 0])
x2_ex = tf.nn.softplus(raw_ex[:, 1])
p_ex = price(x1_ex, x2_ex)

x1_star_ex, x2_star_ex = closed_form_x(c1_ex, c2_ex)
p_star_ex = 1.0 - x1_star_ex - x2_star_ex

print("Example (c1=1.0, c2=1.5)")
print(f"  NN    -> x1={x1_ex.numpy().item():.4f},
      ↪ x2={x2_ex.numpy().item():.4f}, \
        p={p_ex.numpy().item():.4f}")
print(f"  exact-> x1={x1_star_ex.numpy().item():.4f}, \
        x2={x2_star_ex.numpy().item():.4f}, p={p_star_ex.numpy().item():.4f}")

```

Plot x_1, x_2 as a function of c_1 for $c_2 = 1.5$

```

range_c1 = np.arange(0, 3.1, 0.1)[:, None]
c2 = 1.5 * np.ones_like(range_c1)

raw_ex = net(tf.concat([range_c1, c2], axis=1), training=False)
x1_ex = tf.nn.softplus(raw_ex[:, 0])
x2_ex = tf.nn.softplus(raw_ex[:, 1])
p_ex = price(x1_ex, x2_ex)
p_cf = price(closed_form_x(range_c1, c2)[0], closed_form_x(range_c1, c2)[1])
plt.plot(range_c1, p_ex.numpy(), label="neural network")
plt.plot(range_c1, p_cf, label="closed form")
plt.legend();

```

Exercise: change neural network specification to improve the fit between neural network prediction and closed form solution

Notes

- The key idea is purely economic: we take the **derivative of profit w.r.t. own output** with 'GradientTape' and train the net so those derivatives are (on average) zero.
- 'softplus' just ensures $x_i \geq 0$ without if-statements.
- You can change demand/cost by editing the three one-liners: 'price', 'C1', 'C2'.

Course takeaway: simulation as a bridge to machine learning

These lectures show that the gap between classical econometrics and modern machine learning is smaller than it appears. OLS, the workhorse of econometrics, is just a specific loss function minimised by a simple algorithm. Ridge and lasso are OLS with an extra line of code. Neural networks are deeper versions of the same idea.

By building everything from scratch using simulation and optimisation, you develop a type of understanding that no amount of reading about formulas can give. You see directly what the parameters do, how the estimates vary, and what happens when assumptions break down.

Bibliography

Downey, Allen. 2014. *Think Stats: Exploratory Data Analysis in Python*. Green Tea Press.

———. 2023. *Probably Overthinking It*. Green Tea Press. <https://greenteapress.com/wp/probably-overthinking-it/>.

McElreath, Richard. 2020. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. Second. Chapman and Hall/CRC.